

Chapter 12

Databases & SQL

Foundations

Go to the data: tables, keys, filters, NULLs, summaries, joins, subqueries and transactions, taught from zero on the Riverstone Supplies database, in PostgreSQL and MySQL.

Approved · Draft v3

Contents

Chapter 12. Databases & SQL Foundations

Why this matters

In plain English

12.1 What a database actually is

12.2 Meet Riverstone Supplies

12.3 Setting up your SQL laboratory

12.4 Your first query: SELECT and FROM

12.5 WHERE: keeping only the rows you want

12.6 NULL: the value that isn't there

12.7 ORDER BY and LIMIT: sorting and top-N

12.8 Transforming values: CASE, dates, and text

12.9 Summarizing: aggregate functions, GROUP BY, and HAVING

12.10 JOIN: combining tables

12.11 How the database reads your query

12.12 Queries inside queries: subqueries and set operations

12.13 Changing data: INSERT, UPDATE, DELETE, and transactions

12.14 Writing SQL that humans can read

12.15 Putting it all together: Monday morning with the sales head

12.16 The same SQL in MySQL

Common mistakes and how to spot them

In the real world: Riverstone's monthly sales report

Tools

The project: rebuild a real report in SQL

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 12. Databases & SQL Foundations

Part II — The Analyst

Chapter at a glance

You will learn to: explain what a database is and why businesses use one · read a table's structure (columns, data types, keys) and an entity-relationship diagram · write queries that choose, filter, sort, calculate, summarize, and combine data · avoid the classic traps (NULLs, AND/OR, duplicate rows from joins) · answer real business questions (inactive customers, discount leakage, overdue payments, sales performance) step by step · rebuild a real monthly report in SQL.

Before you start: Chapter 1 (what data is), Chapter 3 (how a business runs on data), and Chapters 10–11 (spreadsheets). You do not need any programming experience.

Time needed: 15–18 hours of reading and practice, spread over three weeks.

Tools: PostgreSQL (free database) and DBeaver (free query editor), with MySQL as an optional second database. Setup for both is covered in section 12.3; section 12.16 shows how the chapter's SQL changes in MySQL.

Practice data: the Riverstone Supplies database (Appendix E), in PostgreSQL and MySQL versions. Every query in this chapter was run against it, and every result shown is the real output.

Why this matters

Here is a small, career-defining truth: **the data does not come to you. You go to the data.** And the language you go in is SQL.

In Chapters 10 and 11 you worked with spreadsheets, where the data sits right in front of you and you can see every cell. That works well for thousands of rows. But the sales records of even a mid-sized company run to millions of rows, spread across dozens of connected tables, updated every minute by many people and systems at once. No spreadsheet can hold that, and no human can scroll through it. That data lives in a **database**, and SQL is how you ask the database questions.

SQL is the single highest-leverage skill in this book. It is the one skill shared by every role on the map from Chapter 7: analysts, business analysts, data scientists, data engineers, and architects all write SQL, often every day. It has been one of the most requested skills in data job listings for decades, and it has outlived hundreds of newer tools. And it is learnable. The core that covers most real work fits in this chapter.

By the end of this chapter you will have rebuilt a real monthly sales report in SQL. That is not a toy. It is the first thing many analysts are asked to do in their first job.

In plain English

Imagine a well-run office with a wall of filing cabinets.

- One drawer holds a card for every **customer**: name, city, the kind of business.
- Another drawer holds a card for every **product**: name, category, price.
- A third drawer holds a card for every **order**: which customer placed it, on what date, and its status.
- A fourth holds the **order lines**: which products were in each order, and how many.

Nobody writes the customer's full address on every order card. They write the customer's *number*, and anyone who needs the address looks it up in the customer drawer. That keeps things tidy: if a customer moves, you change one card, not a thousand.

Now imagine a clerk who has memorized where everything is. You don't go to the cabinets yourself. You hand the clerk a note: *"Bring me the total sales for each city last month, biggest first."* The clerk goes off, pulls the right cards, adds things up, and comes back with a neat answer.

- The filing cabinets are the **database**.
- Each drawer is a **table**.
- Each card is a **row**.
- Each field on a card (name, city, price) is a **column**.
- The customer number written on an order card is a **key** that links two drawers.
- The clerk is the **database management system**.
- The note you hand over is a **SQL query**.

Notice what you did *not* do. You didn't tell the clerk which drawer to open first or how to add up the numbers. You described the answer you wanted. That idea, **describe the result, not the steps**, is the heart of SQL.

12.1 What a database actually is

Tables, rows, and columns

A **relational database** stores data in **tables**. A table looks like a spreadsheet tab: it has named **columns** across the top and **rows** of data below. But a database table is much stricter than a spreadsheet, and that strictness is a feature:

Spreadsheet	Database table
Any cell can hold anything: a number, a note, a color	Every column has one fixed data type
You can merge cells, leave random blanks, add a total row in the middle	Every row has the same columns; no merged cells, no stray totals
Rows have no guaranteed identity	Each row is identified by a primary key
Links to other sheets are formulas that can break	Links to other tables are declared relationships the database enforces

Spreadsheet	Database table
One person edits at a time, comfortably	Thousands of users and systems can read and write at once
Strains past a few hundred thousand rows	Handles millions or billions of rows

Data types

Every column is declared with a **data type**, which tells the database what kind of value it may hold. You will meet these constantly:

Type (PostgreSQL name)	Holds	Example
INTEGER	Whole numbers	42 , 5001
NUMERIC (10,2)	Exact decimals, e.g. money; here up to 10 digits with 2 after the point	1450.00
VARCHAR(100)	Text, up to 100 characters	'Metro Mart'
DATE	A calendar date	'2026-02-19'
TIMESTAMP	A date and a time	'2026-02-19 14:30:00'
BOOLEAN	True or false	TRUE

Types matter more than beginners expect. Because `order_date` is a real `DATE`, the database knows that February comes after January and can group dates by month. If someone had stored dates as text, `'10-01-2026'` and `'9-01-2026'` would sort in the wrong order, and nobody could agree whether the first means 10 January or 1 October. When a report looks wrong, a mis-typed column is one of the first things to check.

Real-life example: phone numbers are text, not numbers. A mobile number looks numeric, but store it as `INTEGER` and trouble follows. The leading zero in `022 2345 6789` disappears, `+91` can't be stored at all, and nobody will ever add two phone numbers together. PIN codes, GST numbers, employee codes like `EMP-0042`, and bank account numbers are the same. **Rule of thumb: if you would never do arithmetic on it, store it as text.**

Watch out: money and decimals. Never store money in a "floating-point" type (`REAL`, `FLOAT`, `DOUBLE PRECISION`). Those types store approximations, so `0.1 + 0.2` may come out as `0.30000000000000004`. Use `NUMERIC` / `DECIMAL` for money. Interviewers like this one.

Primary keys: every row needs an identity

A **primary key** is a column (or set of columns) whose value is unique for every row and never empty. In the customers table it's `customer_id`. Two customers might share a name, since there could be two "Metro Mart" shops in different cities, but they can never share a `customer_id`.

Why not just use the name? Names change, get misspelled, and repeat. An ID is stable. That's why nearly every business system you'll meet (ERP, CRM, billing) gives every customer, product, invoice, and employee an ID.

Real-life example: keys you already use. A train ticket's PNR number, an invoice number, a courier tracking number, your employee ID, a car's registration number: each identifies exactly one thing. When a customer-care agent asks for your *order number* instead of your name, they're asking for a primary key, because there may be a hundred customers called Rahul Sharma but only one order 5001.

A good primary key has three properties: it's **unique**, it's **never empty**, and it **never changes**. That's why a mobile number is a poor customer key even though it's usually unique: people change numbers, families share one, and some customers have two.

Sometimes no single column is unique, but a combination is. In a school attendance register, *student* alone repeats every day and *date* repeats for every student, but *student + date* identifies exactly one attendance mark. A key made of several columns is called a **composite key**.

Foreign keys: how tables connect

A **foreign key** is a column in one table that holds the primary key of a row in *another* table. In Riverstone's `orders` table, `customer_id` is a foreign key pointing to `customers.customer_id`. It's the customer number written on the order card. A restaurant bill that says *Table 7* works the same way: the bill doesn't describe the table, it *points* to it.

The database **enforces** the relationship. If someone tries to insert an order for customer 999 and no such customer exists, the database refuses. This is called **referential integrity**, and it's one of the main reasons companies trust databases over spreadsheets.

Relationships

The most common relationship is **one-to-many**: one customer can place many orders, but each order belongs to exactly one customer. You'll also meet:

- **One-to-one**: one employee has one payroll record; one passport belongs to one person.
- **Many-to-many**: one order contains many products, and one product appears in many orders. A database handles this with a **bridge table** (also called a *junction* or *linking* table). At Riverstone, `order_items` is the bridge between `orders` and `products`.

Many-to-many relationships are everywhere once you look for them, and they're always solved the same way, with a table in the middle:

Many...	...to many	Bridge table	One row of the bridge means
Students	Courses	<code>enrolments</code>	this student takes this course
Doctors	Patients	<code>appointments</code>	this doctor sees this patient at this time
Movies	Actors	<code>cast_members</code>	this actor plays this role in this movie
Orders	Products	<code>order_items</code>	this order includes this product, in this quantity

Notice that the bridge table usually carries facts of its own: the appointment time, the actor's role, the quantity ordered. Those facts belong to the *pair*, not to either side alone.

Why split data into many tables?

A beginner's instinct is to put everything in one giant table: every order line with the customer's name, city, and segment, and the product's name, category, and price, repeated on every row. That's exactly what a spreadsheet export looks like. It feels simpler. It causes three problems:

1. **Repetition.** Sharma Hardware's name and city would be copied onto every line of every order they ever placed.
2. **Update errors.** If Sharma Hardware moves from Mumbai to Thane, someone must change hundreds of rows. Miss one and your reports now show the same customer in two cities.
3. **Orphan facts.** You can't record a new customer until they place an order, because there's no row to put them on.

Splitting data into focused tables linked by keys, so that each fact is stored in exactly one place, is called **normalization**. You'll study its formal rules (first, second, and third normal form) in Chapter 28. For now, hold the intuition: **store each fact once, and link to it.**

The price of normalization is that answering a business question often means stitching tables back together. That stitching is called a **join**, and it's the most important skill in this chapter (section 12.10).

Database systems and SQL dialects

The software that runs a database is a **database management system (DBMS)**. Common ones you'll hear about:

- **PostgreSQL, MySQL, Microsoft SQL Server, Oracle, SQLite:** traditional relational databases, often running the business's live systems.
- **Snowflake, Google BigQuery, Amazon Redshift, Databricks SQL:** cloud *data warehouses* built for analysis (Chapter 49).

All of them speak **SQL** (Structured Query Language; say "S-Q-L" or "sequel", both are fine). The core of SQL is standardized, so about 90% of what you learn here works everywhere. The remaining 10%, mostly date functions, text functions, and how to limit rows, differs by system. These variations are called **dialects**. This book uses PostgreSQL as its main database and flags the common differences with a **Dialect note**. It also shows **MySQL**, the other free database you're most likely to meet at work: section 12.3 installs both, and section 12.16 collects every MySQL difference that matters in this chapter.

12.2 Meet Riverstone Supplies

Throughout this book you'll work with the data of **Riverstone Supplies**, a *fictional* company that sells storage boxes, kitchenware, and industrial crates to shops, hotels, and wholesalers across India. Like any real business, it takes orders, bills its customers, and chases payments. (Every name and number

in it is invented.) In this chapter we use a small slice of its database, small enough that you can check every answer by eye. The full-size version, with thousands of orders, is in the companion files (Appendix E).

The schema

A **schema** is the blueprint of a database: which tables exist, their columns, and how they link. Riverstone's mini database has seven tables.

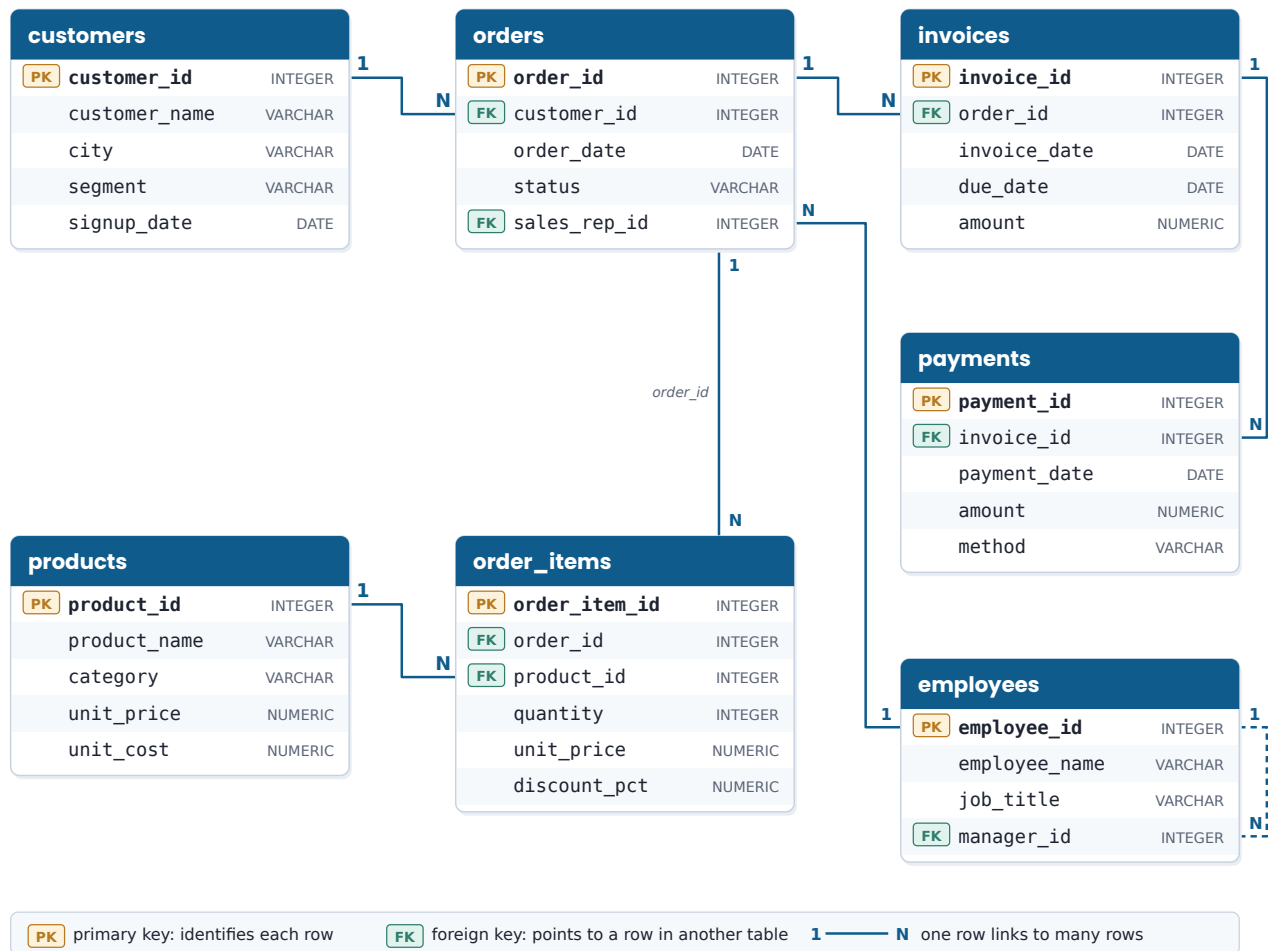


Figure 12.1 – The Riverstone schema, drawn as an entity-relationship diagram.

How to read a schema diagram

Diagrams like Figure 12.1 are called **entity-relationship (ER) diagrams**. Every data team uses them, and you'll be handed one in your first week of almost any data job. Reading one takes three steps:

1. **Each box is a table.** Its name is in the blue header. The rows underneath are its columns, with each column's data type on the right.
2. **Find the keys.** The **PK** badge marks the table's primary key. **FK** badges mark foreign keys: columns that hold another table's primary key.
3. **Follow the lines.** Each line connects a primary key to a foreign key that points at it. The **1** sits next to the table with *one* matching row; the **N** sits next to the table that can have *many*. The

dashed line from `employees` back to itself means an employee's manager is also an employee.

Now read the diagram out loud as sentences. If a sentence sounds wrong for the business, the design is probably wrong:

Line in the diagram	Read it as	What it means in real life
<code>customers</code> → <code>orders</code>	one customer, many orders	Sharma Hardware has placed three orders
<code>employees</code> → <code>orders</code> (<code>sales_rep_id</code>)	one employee, many orders	Neha Kulkarni handles several orders
<code>orders</code> → <code>order_items</code>	one order, many lines	order 5001 contains two different products
<code>products</code> → <code>order_items</code>	one product, many lines	the Water Bottle appears in five orders
<code>orders</code> → <code>invoices</code>	one order, (usually) one invoice	each shipped order is billed
<code>invoices</code> → <code>payments</code>	one invoice, many payments	Coastal Foods paid invoice 9002 in two parts
<code>employees</code> → <code>employees</code> (<code>manager_id</code>)	one manager, many team members	Vikram Singh manages Neha and Rahul

Why is `orders` → `invoices` drawn as one-to-many? Today every order gets exactly one invoice. But businesses sometimes split an invoice (part-shipments) or re-issue one after a correction. Designing for "one or more" costs nothing now and avoids rebuilding the database later. Good database design plans for how the business really works, not just the usual case.

One real order, spread across five tables

The diagram is abstract. Figure 12.2 makes it concrete. On the left is order 5001 the way Sharma Hardware's purchasing manager sees it: a single order slip. On the right is where each piece of that slip actually lives in the database.

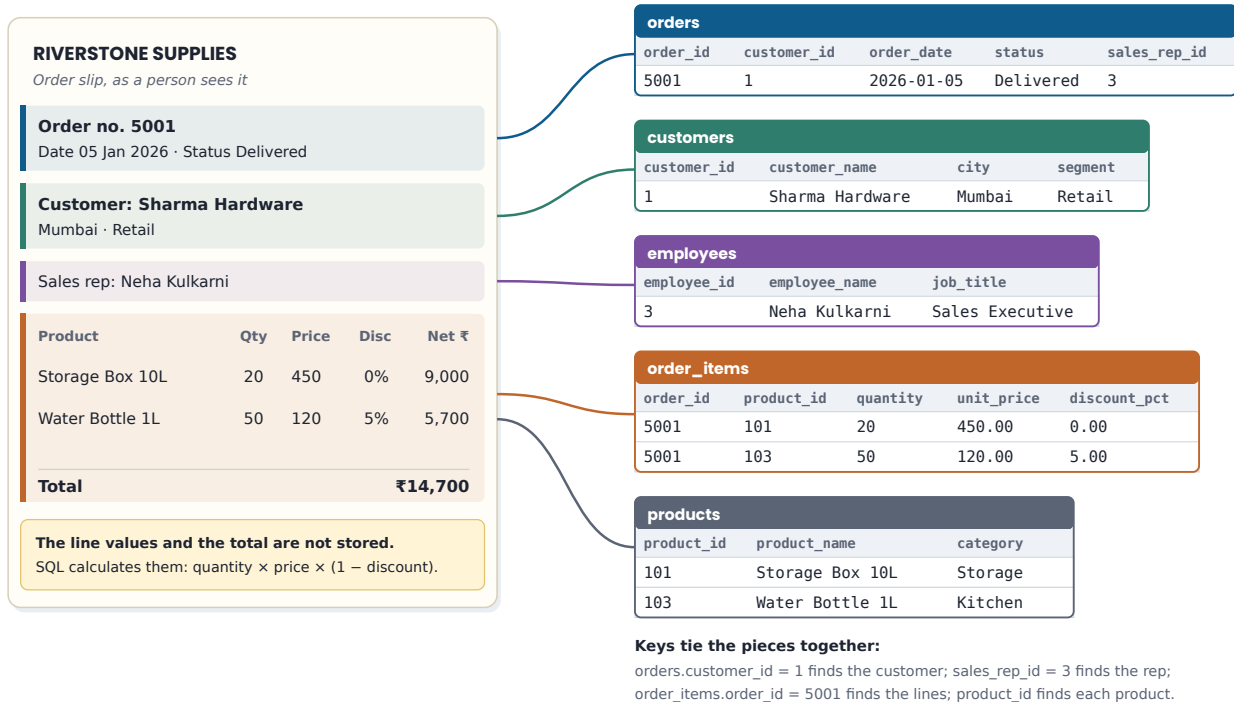


Figure 12.2 – One order slip, five tables. Each colored band on the slip is stored in the table of the same color.

Three things to notice:

1. **Nothing is typed twice.** The slip shows the customer's name, but the `orders` row stores only `customer_id = 1`. The name lives once, in `customers`. If Sharma Hardware renames itself, every past and future order shows the new name automatically.
2. **The slip is a join.** Rebuilding it means combining five tables on their keys. By section 12.10 you'll write that query yourself.
3. **Calculated values aren't stored.** The line values (₹9,000 and ₹5,700) and the total (₹14,700) appear nowhere in the tables. They're calculated from quantity, price, and discount whenever someone asks. A stored total is a second copy of the truth that can drift out of date, for example when someone corrects a discount but forgets the total. (Invoices are the one deliberate exception: a tax invoice is a legal document, so `invoices.amount` records exactly what was billed.)

The data

Here is every row. Take two minutes to read it; you'll understand every result in the chapter faster.

customers

customer_id	customer_name	city	segment	signup_date
1	Sharma Hardware	Mumbai	Retail	2025-11-04
2	Patel Kitchenware	Ahmedabad	Retail	2025-12-15
3	Green Leaf Hotels	Pune	Hospitality	2026-01-08
4	Coastal Foods	Chennai	Wholesale	2025-10-20
5	Metro Mart	Mumbai	Retail	2026-01-22
6	Sunrise Caterers		Hospitality	2026-02-03
7	Northgate Distributors	Delhi	Wholesale	2026-02-17
8	Blue Bay Cafe	Pune	Hospitality	2026-03-01

products

product_id	product_name	category	unit_price	unit_cost
101	Storage Box 10L	Storage	450.00	300.00
102	Storage Box 25L	Storage	780.00	540.00
103	Water Bottle 1L	Kitchen	120.00	70.00
104	Food Container Set	Kitchen	650.00	430.00
105	Industrial Crate	Industrial	1450.00	1100.00
106	Garden Chair	Furniture	1200.00	850.00

employees

employee_id	employee_name	job_title	manager_id
1	Anita Rao	Sales Head	
2	Vikram Singh	Sales Manager	1
3	Neha Kulkarni	Sales Executive	2
4	Rahul Mehta	Sales Executive	2
5	Farah Khan	Sales Executive	1

orders

order_id	customer_id	order_date	status	sales_rep_id
5001	1	2026-01-05	Delivered	3
5002	4	2026-01-09	Delivered	4
5003	2	2026-01-14	Delivered	3
5004	3	2026-01-20	Cancelled	5
5005	1	2026-02-02	Delivered	3
5006	5	2026-02-06	Delivered	4
5007	4	2026-02-11	Delivered	4
5008	6	2026-02-19	Delivered	
5009	7	2026-02-25	Shipped	5
5010	3	2026-03-03	Delivered	5
5011	1	2026-03-10	Shipped	3
5012	5	2026-03-15	Pending	4

order_items

order_item_id	order_id	product_id	quantity	unit_price	discount_pct
1	5001	101	20	450.00	0.00
2	5001	103	50	120.00	5.00
3	5002	105	40	1450.00	10.00
4	5002	102	30	780.00	10.00
5	5003	104	25	650.00	0.00
6	5004	103	100	120.00	0.00
7	5005	101	15	450.00	0.00
8	5005	102	10	780.00	0.00
9	5006	103	60	120.00	5.00
10	5006	104	12	650.00	0.00
11	5007	105	25	1450.00	10.00
12	5008	104	30	650.00	5.00
13	5008	103	40	120.00	0.00
14	5009	105	60	1450.00	12.00
15	5010	102	20	780.00	0.00
16	5010	101	10	450.00	0.00
17	5011	104	18	650.00	0.00
18	5012	101	40	450.00	5.00
19	5012	103	80	120.00	5.00

invoices

invoice_id	order_id	invoice_date	due_date	amount
9001	5001	2026-01-06	2026-02-05	14700.00
9002	5002	2026-01-10	2026-02-09	73260.00
9003	5003	2026-01-15	2026-02-14	16250.00
9004	5005	2026-02-03	2026-03-05	14550.00
9005	5006	2026-02-07	2026-03-09	14640.00
9006	5007	2026-02-12	2026-03-14	32625.00
9007	5008	2026-02-20	2026-03-22	23325.00
9008	5009	2026-02-26	2026-03-28	76560.00
9009	5010	2026-03-04	2026-04-03	20100.00
9010	5011	2026-03-11	2026-04-10	11700.00

payments

payment_id	invoice_id	payment_date	amount	method
1	9001	2026-02-02	14700.00	Bank transfer
2	9002	2026-02-05	40000.00	Bank transfer
3	9002	2026-03-02	33260.00	Bank transfer
4	9003	2026-02-20	10000.00	UPI
5	9004	2026-03-01	14550.00	Cheque
6	9005	2026-03-05	7000.00	UPI
7	9005	2026-03-12	7640.00	UPI
8	9006	2026-03-10	20000.00	Bank transfer
9	9008	2026-03-20	30000.00	Bank transfer
10	9009	2026-03-25	20100.00	UPI

A few details are planted on purpose, because real data always has them:

- **Sunrise Caterers has no city** (an empty value, called `NULL`).
- **Blue Bay Cafe has never ordered.**
- **The Garden Chair has never sold.**
- **Order 5008 has no sales rep.**

- **Order 5004 was cancelled**, so it shouldn't count as revenue.
- `order_items` stores the **price actually charged** at the time of sale, separately from today's list price in `products`. Prices change; history shouldn't.
- `products.unit_cost` is the standard cost of making each product, so you can calculate **profit**, not just revenue.
- Invoices are raised when an order ships (the amount is net of discount; tax is left out to keep the numbers simple). **Customers often pay in parts**: invoices 9002 and 9005 were paid in two instalments, 9003, 9006, and 9008 are only partly paid, and 9007 and 9010 haven't been paid at all.
- Throughout the chapter, "today" is **31 March 2026**, the end of the quarter.

Why store the price twice? It looks like it breaks "store each fact once", but the two columns are *different facts*: the list price today, and the price charged on 5 January. If the Industrial Crate goes up to ₹1,600 next month, January's revenue must not change. Spotting that difference is the kind of judgment interviewers look for.

12.3 Setting up your SQL laboratory

You need two things: a **database server** that stores the data and runs queries, and a **client**, the app where you type queries and see results.

Which database should you install?

This book teaches with **PostgreSQL** and shows **MySQL** alongside it. Both are free, both are used by thousands of companies, and between them they cover most of the databases an analyst meets at work.

	PostgreSQL	MySQL
Where you'll meet it	Analytics teams, fintech, government, SaaS products; the base of several cloud warehouses	Web and e-commerce applications, WordPress sites, many startups' production systems, online stores
Why learn it	Follows the SQL standard closely, so what you learn transfers well	Very likely to be the database behind the app your company runs
In this book	Every query and result is shown in PostgreSQL	Section 12.16 shows every difference that matters, and the companion files have a tested MySQL version of every query

Recommendation: install PostgreSQL first and work through the chapter with it. If your company uses MySQL, or a job description mentions it, install MySQL as well (it takes twenty minutes) and run the queries in both. Seeing the same question answered in two dialects is the fastest way to learn which parts of SQL are universal and which are local spelling. You can run both servers on one computer at the same time; they use different ports.

For both databases you'll use **DBeaver Community Edition** as the client. It's free, runs on Windows, macOS, and Linux, and connects to almost every database, so you learn one editor, not two.

Option A: PostgreSQL

1. **Install PostgreSQL** from the official PostgreSQL website, choosing the installer for your operating system. During setup you'll choose a password for the default `postgres` user. Write it down.
2. **Install DBeaver Community Edition.**
3. **Connect.** In DBeaver, choose *Database → New Database Connection → PostgreSQL*, and use host `localhost`, port `5432`, user `postgres`, and your password. DBeaver offers to download the driver the first time; accept.
4. **Create the database.** Open an SQL editor on that connection and run: `CREATE DATABASE riverstone;` Then edit the connection so its database is `riverstone`, and reconnect.
5. **Load the data.** Open `riverstone_setup.sql` from the companion files (Appendix E) and run the whole script (*Execute SQL Script*, not *Execute Statement*). It creates the seven tables and inserts every row shown above.
6. **Check it worked:** `SELECT COUNT(*) FROM order_items;` should return `19`.

Option B: MySQL

Which version? Install **MySQL Community Server**, the free edition, and choose the current **LTS (Long-Term Support)** release. At the time of writing that's **MySQL 9.7**, released in April 2026 and supported until 2034; **8.4 LTS** is also fine. Avoid **8.0**, which reached end of life in April 2026, and the short-lived "Innovation" releases, which are aimed at people testing new features. Every MySQL query in this book uses features that work the same way in 8.4 and 9.x.

1. Install the server. Download MySQL Community Server from the official MySQL downloads page, then follow the steps for your system:

- **Windows.** Download the **MSI installer** for Windows (64-bit). If the installer asks for the *Microsoft Visual C++ Redistributable*, install that first; the download page links to it. When the MSI finishes, **MySQL Configurator** opens automatically. Accept the *Development Computer* configuration, keep port `3306`, choose a strong password for the `root` account and write it down, and leave *Configure MySQL Server as a Windows Service* and *Start the MySQL Server at System Startup* ticked. You don't need to create extra user accounts or load the sample databases. (Older guides tell you to use "MySQL Installer for Windows". That tool only installs version 8.0 and earlier, so skip it.)
- **macOS.** Download the **DMG archive** that matches your Mac: ARM for Apple silicon (M-series chips), x86 for older Intel Macs. Open it, run the `.pkg` installer, and set the `root` password when asked. MySQL then appears in *System Settings*, where you can start and stop it and choose whether it starts automatically. The command-line client is installed at `/usr/local/mysql/bin/mysql`.
- **Linux (Ubuntu or Debian).** Add Oracle's **MySQL APT repository** by downloading its `mysql-apt-config` package from the MySQL downloads page, then run:

```
sudo dpkg -i mysql-apt-config*_all.deb    # choose the 9.7 LTS (or 8.4 LTS) series when asked
sudo apt-get update
sudo apt-get install mysql-server         # asks you to set the root password
systemctl status mysql                   # should show "active (running)"
```

(Ubuntu's own `apt install mysql-server`, without the Oracle repository, also works but may give you an older series.) Fedora, Red Hat, and similar systems use Oracle's Yum repository in the same way.

2. Connect with DBeaver. Choose *Database* → *New Database Connection* → *MySQL*, and use host `localhost`, port `3306`, user `root`, and your password. Leave *Database* blank for now. Accept the driver download.

Troubleshooting: "Public Key Retrieval is not allowed". This is the most common first-connection error with MySQL 8.4 and later. Modern MySQL protects passwords with an authentication method that needs a public key, and the driver won't fetch it unless you allow it. For a server on your own computer, open the connection's *Driver properties* tab, set `allowPublicKeyRetrieval` to `true` (and `useSSL` to `false` if it still complains), and reconnect. Don't copy these settings to a company server; ask the database administrator for the correct secure connection details instead.

3. Load the data. Open `riverstone_setup_mysql.sql` from the companion files and run the whole script (*Execute SQL Script*). Unlike the PostgreSQL script, this one **creates the riverstone database itself** (`CREATE DATABASE IF NOT EXISTS riverstone; USE riverstone;`), so there's no separate step. Refresh the connection, and `riverstone` appears in the navigator. Double-click it to make it the active database; DBeaver shows the active database in the editor's toolbar.

If you prefer the command line, this does the same thing (you'll be asked for the root password):

```
mysql -u root -p < riverstone_setup_mysql.sql
```

4. Check it worked: run `USE riverstone;` then `SELECT COUNT(*) FROM order_items;`. It should return `19`, exactly as in PostgreSQL.

MySQL Workbench, the other client you'll hear about. Oracle's own free client, **MySQL Workbench**, is common in companies that run MySQL, and many online tutorials use it. It adds visual tools for designing schemas and managing users. It connects the same way (host, port 3306, user, password), and every query in this book runs in it unchanged. Use whichever your team uses; this book's screenshots and tips use DBeaver because it works with both databases.

Words you'll see during MySQL setup. A *schema* in MySQL is the same thing as a *database*: `CREATE SCHEMA riverstone` and `CREATE DATABASE riverstone` are identical. (In PostgreSQL a schema is a folder *inside* a database, which is why the two tools use the word differently.) A *collation* is the set of rules for comparing and sorting text, including whether `'delivered'` equals `'Delivered'`. MySQL's default collation ignores capital letters; section 12.16 shows why that matters.

Can't install software on your computer? Online SQL playgrounds let you paste a setup script and practice in a browser; most offer both PostgreSQL and MySQL. Installation steps and screens change over time; Appendix B keeps current, step-by-step instructions for both databases.

Try it. Before reading on, run `SELECT * FROM products;` and confirm you see six rows. If you installed both databases, run it in both. If you do, your laboratory is ready.

12.4 Your first query: SELECT and FROM

Every query that reads data starts with two clauses:

- `FROM` names the **table** to read.
- `SELECT` names the **columns** you want back.

```
SELECT customer_name, city
FROM customers;
```

Read it aloud: *from the customers table, select the name and city.*

customer_name	city
Sharma Hardware	Mumbai
Patel Kitchenware	Ahmedabad
Green Leaf Hotels	Pune
Coastal Foods	Chennai
Metro Mart	Mumbai
Sunrise Caterers	
Northgate Distributors	Delhi
Blue Bay Cafe	Pune

(8 rows)

A few rules of the language, learned once:

- **Keywords aren't case-sensitive.** `select`, `SELECT`, and `Select` all work. The convention in this book, and in most teams, is to write keywords in capitals so they stand out.
- **The semicolon ;** ends a statement. Some tools don't insist on it, but it's a good habit, especially when you run several queries at once.
- **Line breaks and spaces don't matter** to the database. They matter a great deal to the humans who read your query later, including you.
- **Text values go in single quotes:** `'Mumbai'`. Double quotes mean something different in most databases (a column or table name), so a beginner who writes `"Mumbai"` gets a confusing "column does not exist" error.
- **Comments** start with `--` and run to the end of the line. Use them to explain *why* a query does something.

SELECT * — every column

SELECT * returns all columns:

```
SELECT * FROM products;
```

It's handy for a quick look at an unfamiliar table. In real work, **name your columns**: the query is clearer, it runs faster on wide tables, and it won't silently change when someone adds a column to the table next year.

Calculated columns and aliases

SELECT can compute new values, not just return stored ones. Suppose Riverstone must quote prices including an 18% tax:

```
SELECT product_name,
       unit_price,
       ROUND(unit_price * 1.18, 2) AS price_incl_tax
FROM products;
```

product_name	unit_price	price_incl_tax
Storage Box 10L	450.00	531.00
Storage Box 25L	780.00	920.40
Water Bottle 1L	120.00	141.60
Food Container Set	650.00	767.00
Industrial Crate	1450.00	1711.00
Garden Chair	1200.00	1416.00

(6 rows)

AS price_incl_tax gives the new column a name, called an **alias**. Without it, the database invents an unhelpful name like round. ROUND(value, 2) rounds to two decimal places. The calculation doesn't change anything stored in the table; it exists only in your result.

You can use the usual arithmetic operators: +, -, *, /.

Watch out: integer division. In PostgreSQL and SQL Server, dividing one whole number by another throws away the remainder: 7 / 2 gives 3, not 3.5. When you need a decimal answer, make one side a decimal: 7 / 2.0 or 7 * 1.0 / 2. This quietly breaks percentage calculations in a lot of beginners' reports. (MySQL does the opposite: / always gives a decimal, 3.5000, and DIV gives the whole-number result. Writing 100.0 * works correctly in all of them.)

DISTINCT — unique values only

Which cities do our customers come from?

```
SELECT DISTINCT city
FROM customers
ORDER BY city;
```

```
city
-----
Ahmedabad
Chennai
Delhi
Mumbai
Pune

(6 rows)
```

`DISTINCT` removes duplicate rows from the result: Mumbai and Pune each appear once. Notice the blank sixth row: Sunrise Caterers' missing city counts as a value of its own. More on that in section 12.6.

12.5 WHERE: keeping only the rows you want

`WHERE` is a filter. The database checks the condition against each row and keeps only the rows where it's **true**.

```
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE status = 'Delivered';
```

```
order_id | customer_id | order_date | status
-----+-----+-----+-----
    5001 |           1 | 2026-01-05 | Delivered
    5002 |           4 | 2026-01-09 | Delivered
    5003 |           2 | 2026-01-14 | Delivered
    5005 |           1 | 2026-02-02 | Delivered
    5006 |           5 | 2026-02-06 | Delivered
    5007 |           4 | 2026-02-11 | Delivered
    5008 |           6 | 2026-02-19 | Delivered
    5010 |           3 | 2026-03-03 | Delivered

(8 rows)
```

Comparison operators

Operator	Meaning	Example
<code>=</code>	equal to	<code>status = 'Shipped'</code>
<code><></code> or <code>!=</code>	not equal to	<code>status <> 'Cancelled'</code>
<code>></code> <code><</code>	greater / less than	<code>unit_price > 500</code>
<code>>=</code> <code><=</code>	greater / less than or equal	<code>order_date >= '2026-02-01'</code>

Watch out: text comparisons may be case-sensitive. In PostgreSQL, `'delivered'` does not equal `'Delivered'`. Other databases may ignore case depending on settings. When you're unsure how a column was typed in, compare in a consistent case: `WHERE LOWER(status) = 'delivered'`.

Combining conditions: AND, OR, NOT

- **AND** : both conditions must be true.
- **OR** : at least one must be true.
- **NOT** : reverses a condition.

Here is the classic trap. The sales head asks: "Show me our Retail and Wholesale customers in Mumbai." A beginner writes:

```
SELECT customer_name, city, segment
FROM customers
WHERE segment = 'Retail' OR segment = 'Wholesale' AND city = 'Mumbai';
```

customer_name	city	segment
Sharma Hardware	Mumbai	Retail
Patel Kitchenware	Ahmedabad	Retail
Metro Mart	Mumbai	Retail

(3 rows)

Patel Kitchenware is in Ahmedabad. Why is it here? Because **AND is evaluated before OR**, just as multiplication comes before addition in arithmetic. The database read the condition as:

segment is Retail **OR** (*segment is Wholesale AND city is Mumbai*)

Every Retail customer passed, wherever they were. Parentheses fix it:

```
SELECT customer_name, city, segment
FROM customers
WHERE (segment = 'Retail' OR segment = 'Wholesale')
AND city = 'Mumbai';
```

customer_name	city	segment
Sharma Hardware	Mumbai	Retail
Metro Mart	Mumbai	Retail

(2 rows)

Rule for life: whenever you mix AND and OR, use parentheses, even when you think you don't need them. Notice the wrong query didn't produce an error. It produced a plausible, wrong answer. In analysis, those are the dangerous ones.

IN, BETWEEN, and LIKE

Three shortcuts make filters easier to read.

IN matches any value in a list; it replaces a chain of **OR** s. **BETWEEN** matches a range, and it **includes both ends**.

```
SELECT order_id, order_date, status
FROM orders
WHERE order_date BETWEEN '2026-02-01' AND '2026-02-28'
AND status IN ('Delivered', 'Shipped');
```

```

order_id | order_date | status
-----+-----+-----
    5005 | 2026-02-02 | Delivered
    5006 | 2026-02-06 | Delivered
    5007 | 2026-02-11 | Delivered
    5008 | 2026-02-19 | Delivered
    5009 | 2026-02-25 | Shipped
(5 rows)

```

Watch out: BETWEEN with timestamps. On a `DATE` column, `BETWEEN '2026-02-01' AND '2026-02-28'` is fine. On a `TIMESTAMP` column, `'2026-02-28'` means midnight at the start of 28 February, so every order placed later that day is silently dropped. The safe habit for any date range is a half-open range: `order_date >= '2026-02-01' AND order_date < '2026-03-01'`. It works for dates and timestamps, and for February in leap years.

LIKE matches text patterns. `%` stands for "any number of characters (including none)" and `_` for "exactly one character".

```

SELECT product_name, unit_price
FROM products
WHERE product_name LIKE 'Storage%';

```

```

product_name | unit_price
-----+-----
Storage Box 10L |      450.00
Storage Box 25L |      780.00
(2 rows)

```

Pattern	Matches
'Storage%'	starts with "Storage"
'%Box%'	contains "Box" anywhere
'%L'	ends with "L"
'Storage Box __L'	"Storage Box ", then exactly two characters, then "L"

Dialect note. In PostgreSQL, `LIKE` is case-sensitive; `ILIKE` ignores case. MySQL (with its default collation) and SQL Server (with its usual settings) ignore case with plain `LIKE`, and with `=` too. MySQL has no `ILIKE`.

Real-life example: which products barely make money?

Riverstone's finance manager asks: "Which products earn less than 30% gross margin at list price?"

Gross margin is the share of the selling price left after paying for the product: $(\text{price} - \text{cost}) \div \text{price}$.

```
SELECT product_name,
       unit_price,
       unit_cost,
       ROUND(100.0 * (unit_price - unit_cost) / unit_price, 1) AS margin_pct
FROM products
WHERE (unit_price - unit_cost) / unit_price < 0.30
ORDER BY margin_pct;
```

product_name	unit_price	unit_cost	margin_pct
Industrial Crate	1450.00	1100.00	24.1
Garden Chair	1200.00	850.00	29.2

(2 rows)

How it works, line by line:

- **SELECT ... margin_pct** calculates the margin for display, as a percentage rounded to one decimal. Multiplying by `100.0` rather than `100` keeps the maths in decimals.
- **WHERE (unit_price - unit_cost) / unit_price < 0.30** filters on a *calculation*, not a stored column. `WHERE` can test any expression.
- Notice the `WHERE` repeats the formula instead of writing `WHERE margin_pct < 30`. That's not laziness: the alias `margin_pct` doesn't exist yet when `WHERE` runs. Section 12.11 explains why.
- **ORDER BY margin_pct** puts the weakest product first, and `ORDER BY` can use the alias.

Check one row by hand: the crate sells for ₹1,450 and costs ₹1,100, leaving ₹350, and $350 \div 1,450 = 24.1\%$. ✓ Keep this result in mind. In section 12.9 you'll find that the Industrial Crate is also Riverstone's biggest seller, which makes its thin margin a much bigger story.

12.6 NULL: the value that isn't there

Sunrise Caterers has no city. The cell isn't an empty string or a zero; it holds **NULL**, SQL's marker for "unknown or missing". NULL causes more wrong reports than any other single thing in SQL, so it gets its own section.

NULL is not equal to anything, not even NULL

Try to find customers with no city the obvious way:

```
SELECT customer_name
FROM customers
WHERE city = NULL;
```

```
customer_name
-----
(0 rows)
```

Zero rows, and no error. Here's why. Asking "is the city equal to unknown?" can't be answered true or false; the answer is itself *unknown*. SQL uses **three-valued logic**: a condition can be TRUE, FALSE, or UNKNOWN, and `WHERE` keeps only rows where it's TRUE. Any comparison with NULL (`=`, `<>`, `>`, and

so on) gives UNKNOWN, so nothing passes.

To test for NULL, use the special operators `IS NULL` and `IS NOT NULL` :

```
SELECT customer_name
FROM customers
WHERE city IS NULL;
```

```
customer_name
-----
Sunrise Caterers
(1 row)
```

The silent disappearance

This one fools experienced analysts. List every customer *not* in Mumbai:

```
SELECT customer_name, city
FROM customers
WHERE city <> 'Mumbai';
```

```
customer_name | city
-----+-----
Patel Kitchenware | Ahmedabad
Green Leaf Hotels | Pune
Coastal Foods | Chennai
Northgate Distributors | Delhi
Blue Bay Cafe | Pune
(5 rows)
```

Eight customers, two in Mumbai, and only five came back. **Sunrise Caterers vanished**, because "unknown is not Mumbai" is UNKNOWN, not TRUE. If NULLs should be included, say so explicitly:

```
WHERE city <> 'Mumbai' OR city IS NULL
```

Real-life example: unknown is not zero. Imagine a blank "discount" box on an order form. It could mean *no discount was given*, or *nobody wrote the discount down*. Those are different facts, and a database keeps them apart: `0` means none; `NULL` means unknown. Riverstone's order 5008 has no sales rep. That doesn't mean nobody sold it; it means the record is incomplete. A report that silently drops it loses ₹23,325 of revenue (you'll meet that exact trap in exercise 6).

Try it. Find the order with the missing sales rep: `SELECT order_id FROM orders WHERE sales_rep_id IS NULL`; You should get order 5008. Now try `= NULL` instead and watch it return nothing.

Replacing NULLs with COALESCE

`COALESCE(a, b, c, ...)` returns the first value in its list that isn't NULL. It's the standard way to show a readable label instead of a blank:

```
SELECT customer_name,
       COALESCE(city, 'Unknown') AS city
FROM customers
ORDER BY customer_id;
```

customer_name	city
Sharma Hardware	Mumbai
Patel Kitchenware	Ahmedabad
Green Leaf Hotels	Pune
Coastal Foods	Chennai
Metro Mart	Mumbai
Sunrise Caterers	Unknown
Northgate Distributors	Delhi
Blue Bay Cafe	Pune

(8 rows)

NULL in arithmetic

Any arithmetic involving NULL gives NULL: `5 + NULL` is NULL. If one order line has a missing discount and you compute `quantity * unit_price * (1 - discount_pct / 100)`, that line's revenue becomes NULL. Section 12.8 shows that `SUM` then skips it, so your total is quietly too low. Where a missing number genuinely means zero, write `COALESCE(discount_pct, 0)`. Where it doesn't, the data needs fixing, and Chapter 14 covers how.

NULL rules to memorize

1. Test with `IS NULL` / `IS NOT NULL`, never `= NULL`.
2. `WHERE col <> 'x'` silently drops rows where `col` is NULL.
3. Arithmetic with NULL gives NULL.
4. Aggregate functions (except `COUNT(*)`) ignore NULLs (section 12.8).
5. `DISTINCT` and `GROUP BY` treat all NULLs as one group.
6. Before trusting any filter on a column, ask: *can this column be NULL?*

12.7 ORDER BY and LIMIT: sorting and top-N

Rows in a table have **no guaranteed order**. If you don't sort, the database returns rows in whatever order is fastest for it that day, and that order can change. Any time order matters, say so with `ORDER BY`.

```
SELECT customer_name, segment, signup_date
FROM customers
ORDER BY segment ASC, signup_date DESC;
```

customer_name	segment	signup_date
-----+-----		
Blue Bay Cafe	Hospitality	2026-03-01
Sunrise Caterers	Hospitality	2026-02-03
Green Leaf Hotels	Hospitality	2026-01-08
Metro Mart	Retail	2026-01-22
Patel Kitchenware	Retail	2025-12-15
Sharma Hardware	Retail	2025-11-04
Northgate Distributors	Wholesale	2026-02-17
Coastal Foods	Wholesale	2025-10-20
(8 rows)		

Sorting by several columns works like sorting a phone book: first by segment A→Z (`ASC` , ascending, the default), then *within* each segment by newest signup first (`DESC` , descending).

`LIMIT` keeps only the first *n* rows. Combined with `ORDER BY` , it answers "top-*N*" questions:

```
SELECT product_name, unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 3;
```

product_name	unit_price
-----+	
Industrial Crate	1450.00
Garden Chair	1200.00
Storage Box 25L	780.00
(3 rows)	

Dialect note. `LIMIT 3` works in PostgreSQL, MySQL, SQLite, Snowflake, and BigQuery. SQL Server uses `SELECT TOP 3 ...` ; the SQL standard (and Oracle) uses `FETCH FIRST 3 ROWS ONLY` .

Watch out: ties. If two products shared the third-highest price, `LIMIT 3` would return one of them arbitrarily. "Top 3 with ties" is a real business requirement and a favorite interview follow-up; Chapter 13 solves it properly with `RANK()` .

Dialect note: where NULLs sort. PostgreSQL and Oracle put NULLs *last* in ascending order; MySQL and SQL Server put them *first*. Control it explicitly with `ORDER BY city NULLS LAST` in PostgreSQL and Oracle. MySQL and SQL Server don't support `NULLS LAST` ; there, sort on a NULL test first: `ORDER BY city IS NULL, city` in MySQL (section 12.16).

12.8 Transforming values: CASE, dates, and text

CASE: if-then logic inside a query

`CASE` works like the spreadsheet `IF` function from Chapter 10, but it can check many conditions in order. Riverstone's pricing team wants each product labeled by price band:

```

SELECT product_name,
       unit_price,
       CASE
         WHEN unit_price >= 1000 THEN 'Premium'
         WHEN unit_price >= 500  THEN 'Mid-range'
         ELSE 'Budget'
       END AS price_band
FROM products
ORDER BY unit_price;

```

product_name	unit_price	price_band
Water Bottle 1L	120.00	Budget
Storage Box 10L	450.00	Budget
Food Container Set	650.00	Mid-range
Storage Box 25L	780.00	Mid-range
Garden Chair	1200.00	Premium
Industrial Crate	1450.00	Premium

(6 rows)

`CASE` checks each `WHEN` from top to bottom and **stops at the first match**. The Industrial Crate is also ≥ 500 , but it matched `>= 1000` first. That's why the order of conditions matters: put the most specific ones first. If nothing matches and there's no `ELSE`, the result is `NULL`.

`CASE` is one of the most useful tools you'll learn. You'll use it to group messy categories, build flags (`CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END`), and, combined with aggregates in the next section, to pivot data into columns.

Real-life example: which invoices are overdue?

Every business that sells on credit asks this weekly. Finance wants each invoice labelled by how late it is as of 31 March 2026:

```

SELECT invoice_id,
       due_date,
       CASE
         WHEN due_date >= DATE '2026-03-31' THEN 'Not yet due'
         WHEN DATE '2026-03-31' - due_date <= 30 THEN 'Overdue 1-30 days'
         WHEN DATE '2026-03-31' - due_date <= 60 THEN 'Overdue 31-60 days'
         ELSE 'Overdue 60+ days'
       END AS due_status
FROM invoices
ORDER BY due_date;

```

invoice_id	due_date	due_status
9001	2026-02-05	Overdue 31-60 days
9002	2026-02-09	Overdue 31-60 days
9003	2026-02-14	Overdue 31-60 days
9004	2026-03-05	Overdue 1-30 days
9005	2026-03-09	Overdue 1-30 days
9006	2026-03-14	Overdue 1-30 days
9007	2026-03-22	Overdue 1-30 days
9008	2026-03-28	Overdue 1-30 days
9009	2026-04-03	Not yet due
9010	2026-04-10	Not yet due

(10 rows)

How it works:

- `DATE '2026-03-31'` writes a fixed date directly into the query. Subtracting two dates gives the number of days between them. In a live report you'd use `CURRENT_DATE` (today) instead; a fixed date is used here so your results match the book.
- The `WHEN` conditions are checked **top to bottom**. An invoice 45 days late fails the first two tests and matches the third. Because the first match wins, each condition only has to handle what the earlier ones didn't.
- Grouping amounts into bands like 1–30, 31–60, and 60+ days is called **ageing**, and an ageing report is one of the most common reports in finance.

Now look closely at invoice 9001: *Overdue 31–60 days*. But Sharma Hardware paid it in full on 2 February. **This query is wrong for the business**, even though the SQL is perfect. It knows due dates but not payments. A real overdue report must also look at the `payments` table, and building it properly is the main event of section 12.15. It's a lesson worth learning early: **a technically correct query can still give the wrong business answer if it ignores data that changes the answer.**

Working with dates

Business questions are full of time: *this month*, *last quarter*, *same period last year*. Two PostgreSQL functions do most of the work:

- `EXTRACT(part FROM date)` pulls out one part: `YEAR`, `MONTH`, `DAY`, `DOW` (day of week).
- `DATE_TRUNC('month', date)` rounds a date *down* to the start of its month (or `'week'`, `'quarter'`, `'year'`). This is the standard way to group by month.

```
SELECT order_id,
       order_date,
       EXTRACT(MONTH FROM order_date)      AS month_no,
       DATE_TRUNC('month', order_date)::date AS order_month
FROM orders
WHERE order_id <= 5004;
```

order_id	order_date	month_no	order_month
5001	2026-01-05	1	2026-01-01
5002	2026-01-09	1	2026-01-01
5003	2026-01-14	1	2026-01-01
5004	2026-01-20	1	2026-01-01

(4 rows)

Why prefer `DATE_TRUNC` over `EXTRACT(MONTH ...)` for grouping? Because month number `1` means January of *every* year. Once your data covers more than twelve months, grouping by month number merges January 2026 with January 2027. `DATE_TRUNC` keeps the year.

The `::date` is PostgreSQL shorthand for **casting** (converting) a value to another type; the standard form is `CAST(value AS DATE)`. You can also subtract dates: `DATE '2026-03-10' - DATE '2026-01-05'` returns `64`, the number of days between them.

Dialect note. Date functions are where databases differ most. MySQL uses `DATE_FORMAT(order_date, '%Y-%m-01')` or `YEAR()` / `MONTH()` instead of `DATE_TRUNC`, and `DATEDIFF(later, earlier)` to count days. **Never subtract dates with - in MySQL:** it doesn't raise an error, it returns a meaningless number (section 12.16 shows the trap); SQL Server uses `DATETRUNC(month, order_date)` in recent versions and `DATEFROMPARTS(YEAR(d), MONTH(d), 1)` in older ones; BigQuery uses `DATE_TRUNC(order_date, MONTH)`. The idea transfers; check your database's documentation for the spelling.

Working with text

A handful of text functions cover most cleaning and display work:

Function	Does	Example → Result
<code>UPPER(s)</code> / <code>LOWER(s)</code>	change case	<code>UPPER('pune')</code> → PUNE
<code>TRIM(s)</code>	remove spaces at both ends	<code>TRIM(' Pune ')</code> → Pune
<code>LENGTH(s)</code>	count characters	<code>LENGTH('Pune')</code> → 4
<code>SUBSTRING(s FROM 1 FOR 3)</code>	take part of a string	→ Pun
<code>REPLACE(s, 'a', 'b')</code>	swap text	<code>REPLACE('Box 10L', 'L', ' litre')</code> → Box 10 litre
<code>s1 s2</code> or <code>CONCAT(s1, s2)</code>	join text together	'Riverstone' ' Supplies' (in MySQL use <code>CONCAT</code> ; there means OR)

You'll use these heavily in Chapter 14, where real-world text is full of extra spaces, inconsistent capitals, and typos.

12.9 Summarizing: aggregate functions, GROUP BY, and HAVING

So far every query has returned individual rows. But business questions are almost always about **totals, counts, and averages**: *How many orders? What's our revenue by city? Which customers buy most?* Answering them means collapsing many rows into a few summary rows.

Aggregate functions

An **aggregate function** takes many values and returns one:

Function	Returns
<code>COUNT(*)</code>	number of rows
<code>COUNT(column)</code>	number of rows where that column is not NULL
<code>COUNT(DISTINCT column)</code>	number of different non-NULL values
<code>SUM(column)</code>	total
<code>AVG(column)</code>	average (mean) of non-NULL values

Function	Returns
MIN(column) / MAX(column)	smallest / largest; also works on dates and text

The three flavors of `COUNT` answer different questions, and mixing them up is a classic error:

```
SELECT COUNT(*)           AS total_orders,
       COUNT(sales_rep_id) AS orders_with_rep,
       COUNT(DISTINCT customer_id) AS unique_customers
FROM orders;
```

```
total_orders | orders_with_rep | unique_customers
-----+-----+-----
          12 |              11 |              7
(1 row)
```

- 12 orders in total.
- 11 have a sales rep; order 5008's NULL rep was skipped.
- They came from 7 different customers; Sharma Hardware's three orders count once, and Blue Bay Cafe has none.

Watch out: AVG ignores NULLs. If three products have ratings 4, 5, and NULL, `AVG(rating)` is 4.5, the average of the two known ratings, not 3 (as if the missing one were zero). Usually that's what you want. Sometimes it isn't. Decide deliberately.

GROUP BY: one summary row per group

Aggregates over a whole table give one row. `GROUP BY` gives **one row per group**:

```
SELECT status,
       COUNT(*) AS num_orders
FROM orders
GROUP BY status
ORDER BY num_orders DESC, status;
```

```
status | num_orders
-----+-----
Delivered |      8
Shipped   |      2
Cancelled |      1
Pending   |      1
(4 rows)
```

Picture what the database does: it sorts the twelve orders into piles by status, then counts each pile. If you built pivot tables in Chapter 11, this is the same idea: `GROUP BY` is the pivot table's "Rows" area, and the aggregate is its "Values" area.

Now the most important calculation in this chapter: **revenue**. Each order line's net revenue is quantity × price charged × (1 – discount). Summing lines per order:

```
SELECT order_id,
       ROUND(SUM(quantity * unit_price * (1 - discount_pct / 100)), 2) AS order_revenue
FROM order_items
GROUP BY order_id
ORDER BY order_id
LIMIT 5;
```

order_id	order_revenue
5001	14700.00
5002	73260.00
5003	16250.00
5004	12000.00
5005	14550.00

(5 rows)

Check order 5001 by hand: $20 \times ₹450 \times 1.00 = ₹9,000$, plus $50 \times ₹120 \times 0.95 = ₹5,700$, total ₹14,700. ✓

Always hand-check at least one row of any new calculation. It takes a minute and catches most logic errors.

Real-life example: revenue is not profit

Sales teams love revenue. Owners care about profit. With `unit_cost` you can show both, by category, for all non-cancelled orders:

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue,
       ROUND(SUM(oi.quantity * p.unit_cost), 0) AS product_cost,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)
               - oi.quantity * p.unit_cost), 0) AS gross_profit,
       ROUND(100.0 * SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)
                        - oi.quantity * p.unit_cost)
             / SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 1) AS margin_pct
FROM order_items AS oi
JOIN orders AS o ON oi.order_id = o.order_id
JOIN products AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category
ORDER BY net_revenue DESC;
```

category	net_revenue	product_cost	gross_profit	margin_pct
Industrial	161385	137500	23885	14.8
Storage	81810	57900	23910	29.2
Kitchen	80735	52650	28085	34.8

(3 rows)

This query uses joins, which section 12.10 covers in detail; for now, read `JOIN ... ON` as "bring in the matching rows from that table". Here's what each part does:

- **net_revenue** is the money actually charged: $\text{quantity} \times \text{price charged} \times (1 - \text{discount})$.
- **product_cost** is $\text{quantity} \times \text{the product's cost}$. Cost comes from `products`, which is why that table is joined in.
- **gross_profit** is revenue minus cost, summed across every line in the category.

- **margin_pct** divides total profit by total revenue. Notice it divides two *sums*. Averaging each line's margin would give a different, misleading number, because a ₹50 line would count as much as a ₹50,000 one.

Now read it as a manager would. **Industrial crates bring in half of all revenue (₹161,385 of ₹323,930) but earn the lowest margin, 14.8%.** Kitchen products bring in the least revenue but the most profit. The earlier margin query showed that crates are thin even at list price (24.1%); the 10–12% discounts on crate orders push it down to 14.8%. A sales team rewarded on revenue will keep pushing discounted crates. This one query can start a real conversation about pricing and incentives, which is exactly what analysis is for.

Simplification note. `unit_cost` here is today's standard cost. Real costs change over time, and serious profit reporting stores the cost at the time of sale, just as `order_items` stores the price at the time of sale. Chapter 23 covers how businesses define metrics like margin.

The GROUP BY rule

Once a query has `GROUP BY`, **every column in `SELECT` must either be in the `GROUP BY` or be inside an aggregate function.** This fails:

```
SELECT customer_id, order_date, COUNT(*)
FROM orders
GROUP BY customer_id;
```

```
ERROR: column "orders.order_date" must appear in the GROUP BY clause
or be used in an aggregate function
```

And it should fail. Sharma Hardware has three orders on three dates, so which single `order_date` should appear on its one summary row? The database refuses to guess. Decide what you mean: the first order (`MIN(order_date)`), the latest (`MAX(order_date)`), or one row per customer *and* date (add `order_date` to `GROUP BY`).

Dialect note. Modern MySQL rejects this query too, with error 1055 ("*...not in GROUP BY clause... incompatible with sql_mode=only_full_group_by*"). But very old MySQL versions, and servers where an administrator has switched the `ONLY_FULL_GROUP_BY` setting off, accept it and silently pick an arbitrary date. That's worse than an error, because the result looks fine. If you ever inherit a MySQL report that "works" with a query like this, treat its numbers with suspicion. Write standard SQL even when your database lets you get away with less.

Conditional aggregation: CASE inside SUM

Put `CASE` inside an aggregate and you can count or sum only certain rows, side by side in one result. It's how you build pivot-style reports in SQL:

```
SELECT segment,
       COUNT(*) AS customers,
       SUM(CASE WHEN city = 'Mumbai' THEN 1 ELSE 0 END) AS in_mumbai
FROM customers
GROUP BY segment
ORDER BY segment;
```

```
segment | customers | in_mumbai
-----+-----+-----
Hospitality | 3 | 0
Retail | 3 | 2
Wholesale | 2 | 0
(3 rows)
```

HAVING: filtering groups

Which customers have placed more than one order? You can't use `WHERE` :

```
SELECT customer_id, COUNT(*) AS num_orders
FROM orders
WHERE num_orders > 1
GROUP BY customer_id;
```

ERROR: column "num_orders" does not exist

`WHERE` filters **individual rows before** they're grouped, and at that moment no counts exist yet. To filter the **groups after** aggregation, use `HAVING` :

```
SELECT customer_id, COUNT(*) AS num_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY customer_id;
```

```
customer_id | num_orders
-----+-----
1 | 3
3 | 2
4 | 2
5 | 2
(4 rows)
```

	WHERE	HAVING
Filters	rows	groups
Runs	before GROUP BY	after GROUP BY
Can use aggregates like COUNT(*) ?	No	Yes
Example	WHERE status <> 'Cancelled'	HAVING SUM(revenue) > 50000

You can, and often will, use both in one query: `WHERE` to exclude cancelled orders, then `HAVING` to keep only customers above a revenue threshold. Filtering with `WHERE` whenever possible is also faster, because the database groups fewer rows.

12.10 JOIN: combining tables

This is the concept that separates people who can use a database from people who merely poke at one.

Remember why Riverstone's data is split across tables: to store each fact once. But the sales head doesn't want to see "customer 4". She wants "Coastal Foods". **A join stitches tables back together** by matching a key in one table to a key in another.

INNER JOIN

```
SELECT o.order_id,  
       c.customer_name,  
       o.order_date  
FROM orders AS o  
INNER JOIN customers AS c  
      ON o.customer_id = c.customer_id  
ORDER BY o.order_id  
LIMIT 4;
```

order_id	customer_name	order_date
5001	Sharma Hardware	2026-01-05
5002	Coastal Foods	2026-01-09
5003	Patel Kitchenware	2026-01-14
5004	Green Leaf Hotels	2026-01-20

(4 rows)

Read it aloud: *take each order, find the customer whose `customer_id` matches the order's `customer_id`, and put them side by side.*

- `AS o` and `AS c` are **table aliases**, short nicknames so you don't retype full table names.
- `o.order_id` means "the `order_id` column from the table nicknamed `o`". When two tables share a column name (both have `customer_id`), the prefix is required. Use prefixes on every column in a join anyway; readers shouldn't have to guess where a column comes from.
- `ON` states the **matching condition**.
- The word `INNER` is optional; plain `JOIN` means `INNER JOIN`.

An **inner join keeps only rows that have a match on both sides**. Blue Bay Cafe has no orders, so it could never appear in this result.

Spreadsheet link. In Chapter 10 you used `XLOOKUP` to fetch a customer's name from another sheet, one cell at a time. A join is the same idea done for every row at once, with far stricter rules. Chapter 18 shows the same idea again in Python as `pandas.merge`. Three tools, one concept.

LEFT JOIN: keep everything on the left

Suppose the question is "list every customer and their orders, including customers who haven't ordered."

```
SELECT c.customer_name,  
       o.order_id  
FROM customers AS c  
LEFT JOIN orders AS o  
      ON c.customer_id = o.customer_id  
ORDER BY c.customer_id, o.order_id;
```

customer_name	order_id
Sharma Hardware	5001
Sharma Hardware	5005
Sharma Hardware	5011
Patel Kitchenware	5003
Green Leaf Hotels	5004
Green Leaf Hotels	5010
Coastal Foods	5002
Coastal Foods	5007
Metro Mart	5006
Metro Mart	5012
Sunrise Caterers	5008
Northgate Distributors	5009
Blue Bay Cafe	

(13 rows)

A **left join keeps every row from the left table** (the one after `FROM`), whether or not it has a match. Where there's no match, the right table's columns are filled with `NULL`. Blue Bay Cafe appears with an empty `order_id`.

Notice two more things:

1. **Rows multiplied.** Sharma Hardware appears three times, once per order. A join produces one output row *per matching pair*. Eight customers became thirteen rows. Keep this in mind; it causes the biggest trap in this section.
2. The choice between `INNER` and `LEFT` is a **business decision**, not a technical one. "Revenue by customer" can use an inner join. "Customer list with order counts" must use a left join, or your newest customers (the ones sales most wants to chase) silently vanish from the report.

Figure 12.3 shows both joins side by side on three customers.

customers

customer_id	customer_name
1	Sharma Hardware
2	Patel Kitchenware
8	Blue Bay Cafe

orders

order_id	customer_id
5001	1
5005	1
5011	1
5003	2

INNER JOIN result

4 rows: matches only

customer_name	order_id
Sharma Hardware	5001
Sharma Hardware	5005
Sharma Hardware	5011
Patel Kitchenware	5003

Blue Bay Cafe (customer 8) has no matching order.

Lines show rows whose customer_id values are equal.

INNER JOIN keeps a row only when both sides match.

LEFT JOIN keeps every row from the left table (customers); where there is no match, the right side is filled with NULL.

Sharma Hardware appears three times: one output row per match.

LEFT JOIN result

5 rows: every customer kept

customer_name	order_id
Sharma Hardware	5001
Sharma Hardware	5005
Sharma Hardware	5011
Patel Kitchenware	5003
Blue Bay Cafe	NULL

Figure 12.3 — The same match, two results. An inner join keeps only matched rows; a left join also keeps unmatched left rows and fills the gaps with NULL.

Real-life example: the attendance register. HR has an `employees` list and a `leave_requests` table. "Show leave taken by each employee" with an inner join lists only people who took leave. Anyone with zero days vanishes, and the report can't answer "who hasn't taken a day off all year?", which is often the question HR actually cares about. Whenever the business question contains *every*, *all*, or *including those with none*, reach for a left join.

The anti-join: finding what's missing

A left join plus an `IS NULL` filter finds rows that have **no** match. That answers some of the most valuable questions in business: *customers who never ordered, products that never sold, invoices never paid*.

```
-- Customers who have never placed an order
SELECT c.customer_name
FROM customers AS c
LEFT JOIN orders AS o
      ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

```
customer_name
-----
Blue Bay Cafe
(1 row)
```

```
-- Products that have never been sold
SELECT p.product_name
FROM products AS p
LEFT JOIN order_items AS oi
      ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
```

```
product_name
-----
Garden Chair
(1 row)
```

This pattern is called an **anti-join**. Test the `IS NULL` on a column that can *never* be NULL in a real match, such as the right table's primary key, so you're only catching the rows that found no partner.

RIGHT JOIN, FULL OUTER JOIN, and CROSS JOIN

- **RIGHT JOIN** is a left join viewed from the other side: it keeps every row of the *right* table. A `RIGHT JOIN B` gives the same rows as `B LEFT JOIN A`. Most analysts simply write left joins and list the "keep everything" table first; it reads more naturally.
- **FULL OUTER JOIN** keeps every row from *both* tables, with NULLs wherever either side lacks a match. It's the tool for **reconciliation**: comparing the sales system's list of invoices with the finance system's, and showing what's missing on either side. (MySQL doesn't support it directly; you combine a left and a right join with `UNION`.)
- **CROSS JOIN** has no matching condition. It pairs **every row with every row**. With 3 customer segments and 4 product categories, you get $3 \times 4 = 12$ combinations:

```
SELECT s.segment, p.category
FROM (SELECT DISTINCT segment FROM customers) AS s
CROSS JOIN
      (SELECT DISTINCT category FROM products) AS p
ORDER BY s.segment, p.category
LIMIT 5;
```

```
segment | category
-----+-----
Hospitality | Furniture
Hospitality | Industrial
Hospitality | Kitchen
Hospitality | Storage
Retail      | Furniture
(5 rows)
```

That's useful for building a complete grid (every segment $\times$ every category, including combinations with zero sales) that you then left-join real sales onto. It's dangerous by accident: a cross join of two 100,000-row tables produces ten *billion* rows.

The self-join: a table joined to itself

Riverstone's `employees` table records each person's manager as another `employee_id` in the *same* table. To show each employee next to their manager's name, join the table to itself using two different aliases:

```
SELECT e.employee_name,
       e.job_title,
       m.employee_name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m
      ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

employee_name	job_title	manager_name
Anita Rao	Sales Head	
Vikram Singh	Sales Manager	Anita Rao
Neha Kulkarni	Sales Executive	Vikram Singh
Rahul Mehta	Sales Executive	Vikram Singh
Farah Khan	Sales Executive	Anita Rao

(5 rows)

Think of `e` and `m` as two photocopies of the same list: one read as "employees", the other as "managers". It's a left join so that Anita Rao, who has no manager, still appears. Self-joins turn up in org charts, "customers referred by other customers", and comparing a row to the previous one. (Chapter 13 shows a neater tool for that last case.)

Joining three or more tables

Real questions usually cross several tables. *Revenue by city, excluding cancelled orders* needs `customers` (city), `orders` (status), and `order_items` (money):

```
SELECT COALESCE(c.city, 'Unknown') AS city,
       COUNT(DISTINCT o.order_id) AS orders,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
JOIN order_items AS oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
GROUP BY COALESCE(c.city, 'Unknown')
ORDER BY revenue DESC;
```

city	orders	revenue
Chennai	2	105885
Mumbai	5	81810
Delhi	1	76560
Unknown	1	23325
Pune	1	20100
Ahmedabad	1	16250

(6 rows)

Each `JOIN` adds one table and one matching condition. Build multi-table queries **one join at a time**, running the query after each step and checking the row count makes sense before adding the next.

The fan-out trap: when joins inflate your numbers

This is the most expensive mistake in beginner SQL, and it rarely produces an error. The question: *how many orders has each customer placed?* A reasonable-looking query:

```
SELECT c.customer_name, COUNT(*) AS num_orders
FROM customers AS c
JOIN orders      AS o  ON c.customer_id = o.customer_id
JOIN order_items AS oi ON o.order_id    = oi.order_id
GROUP BY c.customer_name
ORDER BY c.customer_name;
```

customer_name	num_orders
Coastal Foods	3
Green Leaf Hotels	3
Metro Mart	4
Northgate Distributors	1
Patel Kitchenware	1
Sharma Hardware	5
Sunrise Caterers	2

(7 rows)

Sharma Hardware has **three** orders, not five. Sunrise Caterers has **one**, not two. What happened? Joining to `order_items` created one row per *order line*, so an order with two products appears twice, and `COUNT(*)` counted lines, not orders. The data "fanned out" to the finest level of detail in the join.

Two fixes:

1. **Count what you mean:** `COUNT(DISTINCT o.order_id)` counts each order once.
2. **Don't join tables you don't need.** This question never needed `order_items`.

```
SELECT c.customer_name, COUNT(DISTINCT o.order_id) AS num_orders
FROM customers AS c
JOIN orders      AS o  ON c.customer_id = o.customer_id
JOIN order_items AS oi ON o.order_id    = oi.order_id
GROUP BY c.customer_name
ORDER BY c.customer_name;
```

customer_name	num_orders
Coastal Foods	2
Green Leaf Hotels	2
Metro Mart	2
Northgate Distributors	1
Patel Kitchenware	1
Sharma Hardware	3
Sunrise Caterers	1

(7 rows)

The same trap with money

Counting too many orders is embarrassing. Counting money twice can be expensive. Finance asks a simple question: *how much have we invoiced, how much has been paid, and how much is still owed?* Joining invoices to their payments seems natural:

```
SELECT SUM(i.amount)           AS total_invoiced,
       SUM(p.amount)          AS total_paid,
       SUM(i.amount) - SUM(p.amount) AS outstanding
FROM invoices AS i
LEFT JOIN payments AS p ON i.invoice_id = p.invoice_id;
```

```
total_invoiced | total_paid | outstanding
-----+-----+-----
      385610.00 |  197250.00 |   188360.00
(1 row)
```

The real total invoiced is ₹297,710 (just `SELECT SUM(amount) FROM invoices`). This query says ₹385,610, and it nearly **doubles** the amount owed to ₹188,360. A collections team chasing that number would be chasing money customers have already paid. To see why, look at the joined rows for three invoices:

```
SELECT i.invoice_id, i.amount AS invoice_amount, p.payment_id, p.amount AS payment_amount
FROM invoices AS i
LEFT JOIN payments AS p ON i.invoice_id = p.invoice_id
WHERE i.invoice_id IN (9001, 9002, 9007)
ORDER BY i.invoice_id, p.payment_id;
```

```
invoice_id | invoice_amount | payment_id | payment_amount
-----+-----+-----+-----
      9001 |    14700.00 |          1 |    14700.00
      9002 |    73260.00 |          2 |    40000.00
      9002 |    73260.00 |          3 |    33260.00
      9007 |    23325.00 |           |
(4 rows)
```

Invoice 9002 was paid in two parts, so the join produced **two rows, each carrying the full invoice amount of ₹73,260**. Summing that column counts the invoice twice. The same happens to invoice 9005. Payments aren't affected, because each payment appears once: payments are the finest grain in the join.

What goes wrong: join first, then add up

invoices

invoice_id	amount
9002	73260.00

payments

invoice_id	amount
9002	40000.00
9002	33260.00

after the JOIN (2 rows)

invoice_id	invoice_amt	paid
9002	73260.00	40000.00
9002	73260.00	33260.00

SUM(invoice_amt) = 146,520 × counted twice

SUM(paid) = 73,260 ✓ payments are the finest grain, so none repeat

The fix: add up payments first, then join

payments summed per invoice

invoice_id	paid
9002	73260.00

after the JOIN (1 row)

invoice_id	invoice_amt	paid
9002	73260.00	73260.00

One row per invoice: totals are correct ✓

Rule: bring every table to the same grain before joining.

Figure 12.4 – The fix is to bring payments to the invoice's grain (one row per invoice) before joining.

The fix: **add up payments per invoice first**, in a subquery, and join that one-row-per-invoice result:

```

SELECT SUM(i.amount)                AS total_invoiced,
       SUM(COALESCE(p.paid, 0))      AS total_paid,
       SUM(i.amount - COALESCE(p.paid, 0)) AS outstanding
FROM invoices AS i
LEFT JOIN (
  SELECT invoice_id, SUM(amount) AS paid
  FROM payments
  GROUP BY invoice_id
) AS p ON i.invoice_id = p.invoice_id;

```

```

total_invoiced | total_paid | outstanding
-----+-----+-----
297710.00 | 197250.00 | 100460.00
(1 row)

```

How it works:

- The subquery in parentheses runs first and returns **one row per invoice** with its total paid. (Subqueries get a full section in 12.12.)
- The left join now matches each invoice to at most one row, so nothing is repeated.
- `COALESCE(p.paid, 0)` turns "no payments" (NULL) into zero, so unpaid invoices still count in full. Without it, `i.amount - NULL` would be NULL and those invoices would silently drop out of `outstanding`.
- **Reconcile:** ₹297,710 invoiced minus ₹197,250 paid is ₹100,460. The totals now agree with each table summed on its own. ✓

There's no `SUM(DISTINCT ...)` shortcut here, by the way. `SUM(DISTINCT i.amount)` would add each *different amount* once, so two separate invoices that happened to be for the same amount would be counted as one. The only reliable fix is to aggregate to the right grain before joining. Chapter 13 shows a tidier way to write these steps, using CTEs.

The grain question. Before writing any join, ask: *what does one row of each table represent?* One row of `orders` is one order; one row of `order_items` is one product line within an order. That "what is one row" is called the table's **grain**. When you join to a finer grain, your rows multiply. Chapter 28 builds a whole discipline on this idea.

The LEFT JOIN that quietly became an INNER JOIN

Question: *list every customer, and show their Pending orders if they have any.* Two versions that look almost the same:

```

-- Version A: condition in ON
SELECT c.customer_name, o.order_id, o.status
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
 AND o.status = 'Pending'
ORDER BY c.customer_id;

```

customer_name	order_id	status
Sharma Hardware		
Patel Kitchenware		
Green Leaf Hotels		
Coastal Foods		
Metro Mart	5012	Pending
Sunrise Caterers		
Northgate Distributors		
Blue Bay Cafe		

(8 rows)

```
-- Version B: condition in WHERE
SELECT c.customer_name, o.order_id, o.status
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
WHERE o.status = 'Pending'
ORDER BY c.customer_id;
```

customer_name	order_id	status
Metro Mart	5012	Pending

(1 row)

Version A answers the question. Version B throws away every customer without a pending order. Why? In Version B, the join runs first and matches customers to *all* their orders. Then `WHERE` filters that result. Rows for Delivered or Shipped orders fail the test (FALSE), and Blue Bay Cafe's row, which has NULL in every order column, fails too (UNKNOWN). Only Metro Mart's pending order survives. The left join has effectively become an inner join. In Version A, the status test is part of the *matching* rule, so a customer with no pending order simply finds no partner and is kept with NULLs.

Rule: in a `LEFT JOIN`, conditions on the **right-hand** table belong in `ON` if you want to keep unmatched left rows. Conditions on the **left-hand** table go in `WHERE` as usual. This distinction comes up in interviews constantly (Chapter 71).

12.11 How the database reads your query

You write SQL in one order. The database runs it in another. Knowing the difference explains most error messages you'll ever see.

Written order:

```
SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
```

Logical execution order:

- | | |
|----------------|---|
| 1. FROM / JOIN | Gather the tables and combine them into one working set of rows |
| 2. WHERE | Throw away rows that fail the filter |
| 3. GROUP BY | Sort the remaining rows into groups |
| 4. HAVING | Throw away groups that fail the group filter |
| 5. SELECT | Compute the output columns and aliases |
| 6. DISTINCT | Remove duplicate output rows |
| 7. ORDER BY | Sort the result |
| 8. LIMIT | Keep the first n rows |

You write it in this order



The database runs it in this order



This is why WHERE can't use COUNT(*) or a SELECT alias (step 2 runs before steps 3 and 5), and why ORDER BY can use an alias (step 7 runs after step 5).

Figure 12.5 – You write SELECT first, but the database gets to it fifth.

This order explains:

- **Why WHERE can't use COUNT(*)**: at step 2, grouping hasn't happened.
- **Why WHERE can't use a column alias** defined in SELECT, like num_orders: at step 2, SELECT hasn't run, so the alias doesn't exist yet.
- **Why ORDER BY can use an alias**: it runs at step 7, after SELECT created it.
- **Why a LEFT JOIN condition in WHERE removes rows**: the join finished at step 1, and WHERE filters its output at step 2.

(The real database engine is free to rearrange the physical work for speed, but the result always matches this logical order. Chapter 28 shows how to see what it actually does.)

12.12 Queries inside queries: subqueries and set operations

Subqueries

A **subquery** is a query in parentheses used inside another query. Its most common forms:

A single value (scalar subquery). Which products are priced above the average price?

```
SELECT product_name, unit_price
FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products)
ORDER BY unit_price;
```

product_name	unit_price
Storage Box 25L	780.00
Garden Chair	1200.00
Industrial Crate	1450.00

(3 rows)

The inner query runs first and returns one number (₹775.00); the outer query compares every product against it. You couldn't write `WHERE unit_price > AVG(unit_price)`, because aggregates can't appear in `WHERE`.

A list of values, with `IN`. Which customers have bought the Industrial Crate (product 105)?

```
SELECT customer_name
FROM customers
WHERE customer_id IN (
    SELECT o.customer_id
    FROM orders AS o
    JOIN order_items AS oi ON o.order_id = oi.order_id
    WHERE oi.product_id = 105
)
ORDER BY customer_name;
```

customer_name
Coastal Foods
Northgate Distributors

(2 rows)

A yes/no test, with `EXISTS`. Which customers have at least one order currently Shipped?

```
SELECT c.customer_name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
    AND o.status = 'Shipped'
)
ORDER BY c.customer_name;
```

customer_name
Northgate Distributors
Sharma Hardware

(2 rows)

`EXISTS` asks only "is there at least one matching row?", so the `SELECT 1` inside is a convention; what it selects doesn't matter. Notice the inner query refers to `c.customer_id` from the outer query. That makes it a **correlated subquery**: conceptually it re-runs for each customer.

A correlated subquery with a real use. For each invoice, show the most recent payment. It's the kind of thing a collections team asks daily: *when did we last hear from this customer?*

```

SELECT p.invoice_id, p.payment_date, p.amount
FROM payments AS p
WHERE p.payment_date = (
    SELECT MAX(p2.payment_date)
    FROM payments AS p2
    WHERE p2.invoice_id = p.invoice_id
)
ORDER BY p.invoice_id;

```

invoice_id	payment_date	amount
9001	2026-02-02	14700.00
9002	2026-03-02	33260.00
9003	2026-02-20	10000.00
9004	2026-03-01	14550.00
9005	2026-03-12	7640.00
9006	2026-03-10	20000.00
9008	2026-03-20	30000.00
9009	2026-03-25	20100.00

(8 rows)

Read it as: *keep a payment if its date equals the latest payment date for the same invoice*. The inner query uses `p.invoice_id` from the outer row, so for payment 2 it finds the latest date among invoice 9002's payments (2 March), and payment 2 (5 February) is dropped. The table uses two aliases, `p` and `p2`, for the same `payments` table, just like the self-join. Invoices 9007 and 9010 don't appear because they have no payments at all. "Latest record per group" is such a common need that Chapter 13 gives it a cleaner tool, `ROW_NUMBER()`.

Watch out: NOT IN and NULLs. `WHERE customer_id NOT IN (subquery)` returns **no rows at all** if the subquery's list contains even one NULL, because "x is not in (1, 2, NULL)" can't be confirmed true. For "not in" logic, prefer `NOT EXISTS` or the anti-join from section 12.10. This is a favorite interview trap.

A table (derived table) in FROM. A subquery can act as a temporary table you then query. This is how you aggregate at one level and then summarize again. *What's the average order value, excluding cancelled orders?* First total each order, then average those totals:

```

SELECT ROUND(AVG(order_revenue), 2) AS avg_order_value
FROM (
    SELECT o.order_id,
           SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS order_revenue
    FROM orders AS o
    JOIN order_items AS oi ON o.order_id = oi.order_id
    WHERE o.status <> 'Cancelled'
    GROUP BY o.order_id
) AS per_order;

```

avg_order_value
29448.18

(1 row)

Averaging the order *lines* directly would give the average line value, a different and misleading number. Derived tables solve the fan-out problem from section 12.10 too: aggregate each table to the grain you need, then join the results. Once subqueries nest more than one level deep they become hard to read, and Chapter 13 introduces **common table expressions (CTEs)**, which do the same job with named, top-to-bottom steps.

Set operations: stacking results

Joins combine tables *side by side*. Set operations stack results *on top of each other*. The queries being stacked must return the same number of columns, with compatible types.

- **UNION** stacks results and **removes duplicates**.
- **UNION ALL** stacks results and **keeps everything**.

```
SELECT city FROM customers WHERE segment = 'Retail'  
UNION  
SELECT city FROM customers WHERE segment = 'Hospitality'  
ORDER BY city;
```

```
  city  
-----  
Ahmedabad  
Mumbai  
Pune  
  
(4 rows)
```

Replace **UNION** with **UNION ALL** and you get six rows: Ahmedabad once, Mumbai twice, Pune twice, and the blank (NULL) city once. Notice that **UNION** treated NULL like any other value when removing duplicates.

UNION ALL is faster, because removing duplicates requires extra work. Use it by default when you know the pieces don't overlap (January's table stacked on February's) or when duplicates are meaningful. Use **UNION** only when you actually want duplicates removed.

Two more set operations: **INTERSECT** returns rows that appear in both results, and **EXCEPT** (called **MINUS** in Oracle) returns rows in the first result but not the second. **EXCEPT** is a quick reconciliation tool: *which product IDs are in the price list but not in the warehouse system?*

12.13 Changing data: INSERT, UPDATE, DELETE, and transactions

As an analyst you'll mostly *read* data, and in many companies your account will be **read-only**. That's a protection, not an insult. But you must understand how data is written, because you'll create your own practice tables and staging tables, and one day you'll work where you *can* change production data.

Creating a table (you saw this in the setup script):

```
CREATE TABLE target_by_month (
    month          DATE PRIMARY KEY,
    revenue_target NUMERIC(12,2) NOT NULL
);
```

`NOT NULL` forbids missing values; `PRIMARY KEY` enforces uniqueness. These **constraints** are the database protecting data quality for you.

Adding rows:

```
INSERT INTO target_by_month (month, revenue_target)
VALUES ('2026-01-01', 100000),
       ('2026-02-01', 120000);
```

Changing rows:

```
UPDATE orders
SET status = 'Delivered'
WHERE order_id = 5011;
```

Removing rows:

```
DELETE FROM target_by_month
WHERE month = '2026-01-01';
```

Watch out: UPDATE or DELETE without WHERE. `UPDATE orders SET status = 'Delivered';` with no `WHERE` changes **every order in the company**. `DELETE FROM orders;` removes them all. Before running any `UPDATE` or `DELETE`, run the same `WHERE` as a `SELECT` first and check exactly which rows you're about to change.

Transactions: an undo button. A **transaction** groups statements so they succeed or fail together. `BEGIN` starts one; `COMMIT` makes the changes permanent; `ROLLBACK` cancels everything since `BEGIN`.

```
BEGIN;

UPDATE orders SET status = 'Delivered' WHERE order_id = 5011;
SELECT order_id, status FROM orders WHERE order_id = 5011;  -- shows Delivered

ROLLBACK;

SELECT order_id, status FROM orders WHERE order_id = 5011;  -- back to Shipped
```

```
order_id | status
-----+-----
      5011 | Delivered
(1 row)

order_id | status
-----+-----
      5011 | Shipped
(1 row)
```

Transactions are why a bank transfer never takes money out of one account without putting it into the other: both updates commit together, or neither does. That guarantee is part of the **ACID** properties you'll study in Chapter 49.

Dialect note. Some clients, including DBeaver and MySQL Workbench by default, run in **auto-commit** mode: every statement is committed the instant it runs, and there's nothing to roll back. `BEGIN` (or the standard `START TRANSACTION`, which MySQL documentation prefers) switches it off until you `COMMIT` or `ROLLBACK`. Check your client's transaction setting before experimenting with changes.

12.14 Writing SQL that humans can read

A query is read far more often than it's written: by your reviewer, by the colleague who inherits your report, by you in six months. Readable SQL is also *correct* SQL more often, because mistakes have fewer places to hide.

Compare:

```
select c.city,count(distinct o.order_id),sum(oi.quantity*oi.unit_price*(1-oi.discount_pct/100))
from orders o join customers c on o.customer_id=c.customer_id join order_items oi on
o.order_id=oi.order_id where o.status<>'Cancelled' group by c.city order by 3 desc
```

with the version in section 12.10. Same logic. One of them you can review in ten seconds.

A simple style guide:

1. **Capitalize keywords** (`SELECT`, `FROM`, `JOIN`); keep table and column names lower-case.
2. **One major clause per line**, and one column per line when there are more than two or three.
3. **Indent** join conditions and subqueries.
4. **Alias every calculated column** with a clear business name (`net_revenue`, not `x` or `sum`).
5. **Use meaningful table aliases** (`c` for customers, `oi` for order_items), and prefix every column in a multi-table query.
6. **Name columns in `ORDER BY` and `GROUP BY`** rather than using position numbers like `ORDER BY 3`; numbers break silently when someone reorders the `SELECT`.
7. **Comment the *why*, not the *what***: `-- exclude cancelled orders: finance counts revenue on delivery or shipment`.
8. **Name tables and columns in `snake_case`**: lower-case words joined by underscores.

Teams often adopt a published SQL style guide and enforce it with an automatic formatter ("linter"). You'll meet that in Chapter 32.

12.15 Putting it all together: Monday morning with the sales head

It's Monday, 31 March 2026, the last day of the quarter. Before the 10 a.m. review, Anita Rao, Riverstone's Sales Head, sends you four questions. None of them mentions SQL. This is what real analysis work looks like, so we'll answer each one the way an experienced analyst would, with a method rather than guesswork.

A six-step method for any business question

1. **Restate the question precisely.** What exactly is being counted? As of when? Which records are included or excluded (cancelled orders, unpaid invoices)? Write the business rules down.
2. **Decide the shape of the answer.** What does one row of the result represent: one customer, one month, one invoice? Which columns will the reader need?
3. **Find the tables.** Which tables hold each piece? What is the **grain** (one row = what?) of each?
4. **Plan the joins.** Which keys connect them? Inner or left join? Will any join multiply rows (fan-out)?
5. **Build in small steps.** Start with one table, add one join at a time, then filters, then grouping. Run the query after every step and check the row count.
6. **Check the answer.** Hand-check one row. Reconcile a total to a number you already trust. Then ask: *does this make business sense?*

Question 1: "Which customers have gone quiet?"

Restate. Customers with no order in the 30 days up to 31 March 2026, meaning no order since 1 March. Cancelled orders don't count as activity. Customers who have *never* ordered must be included, because they're the easiest to miss.

Shape. One row per customer: name, segment, last order date, days since that order.

Tables and joins. `customers` and `orders`. It must be a **left join**, or never-ordered customers disappear. The "not cancelled" rule goes in the `ON` clause, for the reason you saw in section 12.10.

Filter. "Last order before 1 March" is a condition on `MAX(order_date)`, an aggregate, so it belongs in `HAVING`, not `WHERE`.

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date)           AS last_order_date,
       DATE '2026-03-31' - MAX(o.order_date) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
 AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < DATE '2026-03-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date NULLS FIRST;
```

customer_name	segment	last_order_date	days_since_last_order
Blue Bay Cafe	Hospitality		
Patel Kitchenware	Retail	2026-01-14	76
Coastal Foods	Wholesale	2026-02-11	48
Sunrise Caterers	Hospitality	2026-02-19	40
Northgate Distributors	Wholesale	2026-02-25	34

(5 rows)

How it works. `GROUP BY c.customer_id, ...` makes one group per customer (grouping by the ID as well as the name protects against two customers sharing a name). `MAX(o.order_date)` finds each customer's latest valid order; for Blue Bay Cafe there are no matching orders, so it's NULL. `HAVING ... OR MAX(o.order_date) IS NULL` keeps the never-ordered customers, because "NULL < 1 March" is unknown, not true (section 12.6). `NULLS FIRST` puts them at the top.

Check. Green Leaf Hotels isn't listed. Its January order was cancelled, but it ordered again on 3 March, so it's active. ✓ Sharma Hardware (10 March) and Metro Mart (15 March) are active too. ✓

What to tell Anita. Five of eight customers haven't ordered this month. Blue Bay Cafe signed up on 1 March and has never ordered: a warm lead to call today. Patel Kitchenware has gone 76 days, far longer than any other customer.

Question 2: "How much are discounts costing us?"

Restate. For non-cancelled orders, compare what products would have sold for at the price charged before discount (the *list value*) with the discount given, by category.

Shape. One row per category. **Tables:** `order_items` (quantities, prices, discounts), `products` (category), `orders` (status). Every join goes from a line to its one order and one product, so nothing fans out.

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price), 0)           AS list_value,
       ROUND(SUM(oi.quantity * oi.unit_price * oi.discount_pct / 100), 0) AS discount_given,
       ROUND(100.0 * SUM(oi.quantity * oi.unit_price * oi.discount_pct / 100)
             / SUM(oi.quantity * oi.unit_price), 1)         AS discount_pct_of_list
FROM order_items AS oi
JOIN orders      AS o ON oi.order_id = o.order_id
JOIN products    AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category
ORDER BY discount_given DESC;
```

category	list_value	discount_given	discount_pct_of_list
Industrial	181250	19865	11.0
Storage	85050	3240	3.8
Kitchen	82850	2115	2.6

(3 rows)

Check. List value minus discount should equal the net revenue from section 12.9: for Industrial, 181,250 – 19,865 = 161,385. ✓

What to tell Anita. Riverstone gave away ₹19,865 on crates this quarter, 11% of their value, against under 4% elsewhere. Put that next to the gross profit query from section 12.9: crates earned only ₹23,885 of gross profit. **The discounts on crates were worth about 83% of the profit the crates actually made.** That's a finding worth a meeting.

Question 3: "Who owes us money, and how late is it?"

This is the classic **receivables ageing report**, and it pulls together almost everything in this chapter.

Restate. As of 31 March 2026, for each customer: the total unpaid balance, split by how overdue it is (not yet due, 1–30, 31–60, 60+ days past the due date). Only invoices with money still owed.

Shape. One row per customer, with one column per ageing band.

Tables and grain. `invoices` (one row per invoice), `payments` (one row per payment, **many per invoice**), `orders` (to reach the customer), `customers` (names). The payments join would fan out, so payments must be summed per invoice first.

Build in steps. First, the balance for every invoice:

```
SELECT i.invoice_id,
       i.amount,
       COALESCE(pay.paid, 0.00) AS paid,
       i.amount - COALESCE(pay.paid, 0.00) AS balance
FROM invoices AS i
LEFT JOIN (
  SELECT invoice_id, SUM(amount) AS paid
  FROM payments
  GROUP BY invoice_id
) AS pay ON i.invoice_id = pay.invoice_id
ORDER BY i.invoice_id;
```

invoice_id	amount	paid	balance
9001	14700.00	14700.00	0.00
9002	73260.00	73260.00	0.00
9003	16250.00	10000.00	6250.00
9004	14550.00	14550.00	0.00
9005	14640.00	14640.00	0.00
9006	32625.00	20000.00	12625.00
9007	23325.00	0.00	23325.00
9008	76560.00	30000.00	46560.00
9009	20100.00	20100.00	0.00
9010	11700.00	0.00	11700.00

(10 rows)

Invoice 9001, which section 12.8 wrongly flagged as overdue, now correctly shows a zero balance. Next, wrap that result as a derived table, join it to `orders` and `customers`, and use conditional aggregation (section 12.9) to spread balances into ageing columns:

```

SELECT c.customer_name,
       SUM(inv.balance) AS total_due,
       SUM(CASE WHEN inv.due_date >= DATE '2026-03-31' THEN inv.balance ELSE 0.00 END) AS
not_yet_due,
       SUM(CASE WHEN inv.due_date < DATE '2026-03-31'
                AND DATE '2026-03-31' - inv.due_date <= 30 THEN inv.balance ELSE 0.00 END) AS
overdue_1_30,
       SUM(CASE WHEN DATE '2026-03-31' - inv.due_date BETWEEN 31 AND 60 THEN inv.balance ELSE 0.00
END) AS overdue_31_60,
       SUM(CASE WHEN DATE '2026-03-31' - inv.due_date > 60 THEN inv.balance ELSE 0.00 END) AS
overdue_60_plus
FROM (
  SELECT i.invoice_id, i.order_id, i.due_date,
         i.amount - COALESCE(pay.paid, 0) AS balance
  FROM invoices AS i
  LEFT JOIN (SELECT invoice_id, SUM(amount) AS paid FROM payments GROUP BY invoice_id) AS pay
    ON i.invoice_id = pay.invoice_id
) AS inv
JOIN orders AS o ON inv.order_id = o.order_id
JOIN customers AS c ON o.customer_id = c.customer_id
WHERE inv.balance > 0
GROUP BY c.customer_name
ORDER BY total_due DESC;

```

customer_name	total_due	not_yet_due	overdue_1_30	overdue_31_60	overdue_60_plus
Northgate Distributors	46560.00	0.00	46560.00	0.00	0.00
Sunrise Caterers	23325.00	0.00	23325.00	0.00	0.00
Coastal Foods	12625.00	0.00	12625.00	0.00	0.00
Sharma Hardware	11700.00	11700.00	0.00	0.00	0.00
Patel Kitchenware	6250.00	0.00	0.00	6250.00	0.00

(5 rows)

How it works.

- The innermost subquery totals payments per invoice (the fan-out fix). The next layer computes each invoice's `balance`. The outer query joins balances to customers and aggregates.
- `WHERE inv.balance > 0` removes fully paid invoices *before* grouping.
- Each `SUM(CASE WHEN ... THEN inv.balance ELSE 0.00 END)` adds up only the balances that fall into one ageing band. Five `CASE` columns turn one list of invoices into a five-column report, the same result you'd build with a pivot table in Chapter 11.
- `ELSE 0.00` (rather than leaving out the `ELSE`) makes empty bands show zero instead of `NULL`, which matters when someone totals the columns in Excel.

Check. The `total_due` column adds up to $46,560 + 23,325 + 12,625 + 11,700 + 6,250 = ₹100,460$, exactly the outstanding total from section 12.10. ✓ And each row's bands add up to its total. ✓

What to tell Anita. ₹88,760 is overdue, and more than half of it is Northgate Distributors (₹46,560). Northgate is *also* on the "gone quiet" list from Question 1, so before sales chases them for a new order, finance should chase the old one. Sunrise Caterers hasn't paid anything, has no city recorded, and has no sales rep assigned: a credit risk and a data-quality problem at the same time.

Question 4: "How is each sales rep doing, and who do they report to?"

Restate. For each sales rep: number of orders and net revenue on non-cancelled orders, with their manager's name.

Tables and joins. `employees` twice (the rep, and the rep's manager, a self-join), `orders`, `order_items`. The manager join is a left join, so a rep with no manager still appears.

```
SELECT e.employee_name AS sales_rep,
       COALESCE(m.employee_name, '(none)') AS reports_to,
       COUNT(DISTINCT o.order_id) AS orders,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.employee_id
JOIN orders AS o ON o.sales_rep_id = e.employee_id
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY e.employee_name, m.employee_name
ORDER BY net_revenue DESC;
```

sales_rep	reports_to	orders	net_revenue
Rahul Mehta	Vikram Singh	4	146745
Farah Khan	Anita Rao	2	96660
Neha Kulkarni	Vikram Singh	4	57200

(3 rows)

How it works. `COUNT(DISTINCT o.order_id)` avoids the fan-out from joining to order lines. Both names are in `GROUP BY` because both appear in `SELECT`. Anita Rao and Vikram Singh don't appear as reps because they have no orders of their own; the inner join to `orders` removes them, which is what this question wants.

Check. The three reps total ₹300,605. Order 5008 has no rep and is worth ₹23,325. $300,605 + 23,325 = ₹323,930$, exactly the total non-cancelled revenue. ✓ **Reconciling to a known total is how you prove a report hasn't lost anything.** Mention the unassigned ₹23,325 in a footnote so nobody wonders where it went.

What to tell Anita. Rahul Mehta leads on revenue with ₹146,745, largely from two big Coastal Foods orders. Neha Kulkarni handled as many orders but at a much smaller average size, which is worth understanding before judging performance. Ranking reps, showing each one's share of team revenue, and comparing this quarter with last all need **window functions**, the headline tool of Chapter 13.

What just happened

Four questions, four queries, and each answer came with a number *and* a reason to act. None of the SQL was new; every piece came from sections 12.4 to 12.12. What made the difference was the method: restating the question, knowing the grain of each table, choosing joins deliberately, building in steps, and checking every result against something you already trusted. That habit, far more than syntax, is what makes an analyst trusted.

12.16 The same SQL in MySQL

Everything you've learned in this chapter works in MySQL: tables and keys, `SELECT`, `WHERE`, `NULL` logic, `CASE`, aggregates, `GROUP BY` and `HAVING`, every kind of join except one, subqueries, `UNION`, and transactions. The companion file `ch12_queries_mysql.sql` contains every query in this chapter rewritten for MySQL, and each one was run against `riverstone_setup_mysql.sql` and checked against the PostgreSQL result: **the answers are identical**.

What changes is a short list of spellings, and a shorter list of *behaviors*. The spellings are easy: MySQL tells you with an error. The behaviors are the dangerous part, because MySQL quietly returns a different answer. This section covers both.

The differences that matter in this chapter

Task	PostgreSQL (this chapter)	MySQL
Days between two dates	<code>DATE '2026-03-31' - due_date</code>	<code>DATEDIFF(DATE '2026-03-31', due_date)</code>
Start of the month	<code>DATE_TRUNC('month', order_date)::date</code>	<code>CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE)</code>
Convert a type	<code>value::date</code> or <code>CAST(value AS DATE)</code>	<code>CAST(value AS DATE)</code> only
Join text	<code>a b</code> or <code>CONCAT(a, b)</code>	<code>CONCAT(a, b)</code> only
Case-insensitive match	<code>LOWER(col) = 'x'</code> or <code>ILIKE</code>	<code>=</code> and <code>LIKE</code> already ignore case (default collation)
Put NULLs first or last	<code>ORDER BY col NULLS FIRST / NULLS LAST</code>	<code>ORDER BY col IS NULL DESC, col / ORDER BY col IS NULL, col</code>
NULLs in a plain ascending sort	last	first
<code>7 / 2</code>	3 (whole numbers)	3.5000 (always decimal); <code>7 DIV 2</code> gives 3
Keep every row from both tables	<code>FULL OUTER JOIN</code>	not supported; <code>LEFT JOIN ... UNION ALL ... RIGHT JOIN</code>
Start a transaction	<code>BEGIN;</code>	<code>START TRANSACTION;</code> (or <code>BEGIN;</code>)
Money type	<code>NUMERIC(10,2)</code>	<code>DECIMAL(10,2)</code> (<code>NUMERIC</code> also accepted)
Whole numbers	<code>INTEGER</code>	<code>INT</code> (<code>INTEGER</code> also accepted)
A database vs. a schema	a schema is a folder inside a database	<code>SCHEMA</code> and <code>DATABASE</code> mean the same thing

Everything else in the chapter, including `LIMIT`, `COALESCE`, `EXTRACT`, `COUNT(DISTINCT ...)`, `EXISTS`, `UNION`, `INTERSECT`, and `EXCEPT` (MySQL 8.0.31 and later), is written the same way.

Trap 1: subtracting dates gives a number, not days

In PostgreSQL, subtracting one date from another gives the number of days between them. MySQL accepts exactly the same expression without complaint and returns something else:

```
SELECT DATE '2026-03-31' - DATE '2026-02-05' AS looks_like_days,
       DATEDIFF(DATE '2026-03-31', DATE '2026-02-05') AS actual_days;
```

```
+-----+-----+
| looks_like_days | actual_days |
+-----+-----+
|           126 |          54 |
+-----+-----+
```

MySQL turned both dates into the numbers 20260331 and 20260205 and subtracted them. There are 54 days between 5 February and 31 March, not 126. Imagine this inside the receivables ageing report from section 12.15: invoices only a few weeks late would land in the "60+ days" column, and finance would start chasing customers who aren't seriously late at all.

Rule for MySQL: count days with DATEDIFF(later_date, earlier_date) , always. Notice the order of the arguments: the later date comes first, just as it does in a subtraction.

Here is Question 1 from section 12.15, "Which customers have gone quiet?", in MySQL. Two lines change: DATEDIFF replaces the subtraction, and the sort puts NULLs first by hand, because MySQL has no NULLS FIRST .

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date) AS last_order_date,
       DATEDIFF(DATE '2026-03-31', MAX(o.order_date)) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
  AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < DATE '2026-03-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date IS NULL DESC, last_order_date;
```

```
+-----+-----+-----+-----+
| customer_name | segment | last_order_date | days_since_last_order |
+-----+-----+-----+-----+
| Blue Bay Cafe | Hospitality | NULL | NULL |
| Patel Kitchenware | Retail | 2026-01-14 | 76 |
| Coastal Foods | Wholesale | 2026-02-11 | 48 |
| Sunrise Caterers | Hospitality | 2026-02-19 | 40 |
| Northgate Distributors | Wholesale | 2026-02-25 | 34 |
+-----+-----+-----+-----+
```

The same five customers, in the same order, with the same day counts as the PostgreSQL version. Two details:

- **How the sort works.** last_order_date IS NULL is 1 (true) for Blue Bay Cafe and 0 (false) for everyone else. Sorting that DESC puts the 1s first. The second sort column then orders the rest by date.
- **How MySQL shows results.** The MySQL command-line client draws borders with +, -, and |, and prints missing values as the word NULL rather than a blank. DBeaver shows [NULL] in both databases. It's still a missing value, not the text "NULL".

Trap 2: MySQL ignores capital letters when comparing text

Run this in both databases:

```
SELECT COUNT(*) AS delivered_orders
FROM orders
WHERE status = 'delivered';
```

```
+-----+
| delivered_orders |
+-----+
|                8 |
+-----+
```

PostgreSQL returns **0**, because `'delivered'` and `'Delivered'` are different text. MySQL returns **8**. Its default **collation** (the rules for comparing text, section 12.3) is *case-insensitive*, and also *accent-insensitive*, so `'cafe'` matches `'Café'` too.

That's often convenient: a user who types "mumbai" in a search box still finds Mumbai. But it has real consequences:

- **A query that works in MySQL can silently return nothing when moved to PostgreSQL, Snowflake, or most other warehouses.** When a company migrates its reports, this is one of the first things to break.
- **Duplicate checks behave differently.** In MySQL, `GROUP BY email` treats `Ravi@Example.com` and `ravi@example.com` as one group, and a `UNIQUE` column won't accept both. In PostgreSQL they're two.
- **When you genuinely need an exact match** in MySQL, such as for case-sensitive codes or passwords, ask for a binary comparison: `WHERE status COLLATE utf8mb4_bin = 'delivered'` returns 0.

Portable habit: match the stored capitalization exactly, or write `LOWER(status) = 'delivered'`, which gives the same answer in every database.

Trap 3: `||` means OR, not "join text"

```
SELECT 'Riverstone' || 'Supplies' AS pipes,
       CONCAT('Riverstone', 'Supplies') AS concat_result;
```

```
+-----+-----+
| pipes | concat_result |
+-----+-----+
|      0 | Riverstone Supplies |
+-----+-----+
```

In MySQL's default settings, `||` is an old synonym for `OR`. MySQL tried to treat both pieces of text as true/false values and returned `0`, with only a warning. Recent versions mark this use of `||` as deprecated, but it still runs. **In MySQL, join text with `CONCAT`**, which also works in PostgreSQL, SQL Server, Snowflake, and BigQuery.

Where NULLs sort

`SELECT DISTINCT city FROM customers ORDER BY city;` returns the same six rows in both databases, but PostgreSQL puts Sunrise Caterers' missing city **last**, and MySQL puts it **first**:

```
+-----+
| city   |
+-----+
| NULL   |
| Ahmedabad |
| Chennai |
| Delhi  |
| Mumbai |
| Pune   |
+-----+
```

Neither is wrong; the SQL standard lets each database choose. It matters when a report takes "the first row" or when two people compare exports line by line. To get PostgreSQL's order in MySQL, write `ORDER BY city IS NULL, city`. Better still, decide where missing values belong and say so explicitly in every database.

No FULL OUTER JOIN: reconciliation the MySQL way

Section 12.10 described `FULL OUTER JOIN` as the reconciliation tool. Here's a real use in PostgreSQL: *which orders have no invoice, and which invoices have no order?*

```
SELECT o.order_id, o.status, i.invoice_id
FROM orders AS o
FULL OUTER JOIN invoices AS i ON o.order_id = i.order_id
WHERE o.order_id IS NULL OR i.invoice_id IS NULL
ORDER BY o.order_id;
```

```
order_id | status   | invoice_id
-----+-----+-----
    5004 | Cancelled |
    5012 | Pending  |
(2 rows)
```

MySQL doesn't support `FULL OUTER JOIN`. Build it from its two halves: a left join that finds orders with no invoice, stacked on a right join that finds invoices with no order.

```
SELECT o.order_id, o.status, i.invoice_id
FROM orders AS o
LEFT JOIN invoices AS i ON o.order_id = i.order_id
WHERE i.invoice_id IS NULL
UNION ALL
SELECT o.order_id, o.status, i.invoice_id
FROM orders AS o
RIGHT JOIN invoices AS i ON o.order_id = i.order_id
WHERE o.order_id IS NULL
ORDER BY order_id;
```

order_id	status	invoice_id
5004	Cancelled	NULL
5012	Pending	NULL

The same answer. The cancelled order and the pending order have no invoice, which is correct, and no invoice is orphaned. If an invoice with no matching order ever appeared here, it would mean money billed against an order the sales system doesn't know about: exactly what an audit wants to find.

Each half has its own `WHERE`, so the two halves can never return the same row, which is why `UNION ALL` is safe (and faster than `UNION`). If you want *all* rows from both sides, not just the unmatched ones, remove the first `WHERE`, keep the second, and still use `UNION ALL`: the second half then adds only the rows the first half couldn't produce.

The monthly report, in MySQL

The "In the real world" report at the end of this chapter needs one change: `DATE_FORMAT` builds the first day of each month, because MySQL has no `DATE_TRUNC`.

```
-- Monthly sales summary (MySQL)
-- Revenue counts Delivered and Shipped orders (finance policy).
SELECT CAST(DATE_FORMAT(o.order_date, '%Y-%m-01') AS DATE) AS month,
       COUNT(DISTINCT o.order_id) AS orders,
       COUNT(DISTINCT o.customer_id) AS active_customers,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue
FROM orders AS o
JOIN order_items AS oi
  ON o.order_id = oi.order_id
WHERE o.status IN ('Delivered', 'Shipped')
GROUP BY month
ORDER BY month;
```

month	orders	active_customers	net_revenue
2026-01-01	3	3	104210
2026-02-01	5	5	161700
2026-03-01	2	2	31800

`'%Y-%m-01'` is a format pattern: `%Y` is the four-digit year, `%m` the two-digit month, and `-01` is typed literally, giving text like `2026-02-01`. `CAST(... AS DATE)` turns that text back into a real date, so it sorts and joins like one. The format letters are MySQL's own; `%d` is the day, `%M` the month's name, and `%b` its short name, which is handy for chart labels (`DATE_FORMAT(order_date, '%b %Y')` gives `Feb 2026`).

Habits that make your SQL portable

You'll probably work with more than one database in your career; many companies run MySQL for their applications and copy the data into a warehouse for analysis. These habits make a query move between them with the fewest surprises:

1. **Count days with a function** (`DATEDIFF` in MySQL), never by subtracting dates.
2. **Join text with** `CONCAT` .
3. Use `CAST(x AS type)` , not `::` .
4. **Match text exactly, or use** `LOWER()` **on purpose**. Never rely on a database ignoring case.
5. **Say where NULLs sort** whenever the order of rows matters.
6. **Write** `100.0 *` in percentage calculations, so division behaves the same everywhere.
7. **Keep every non-aggregated column in** `GROUP BY` , even if a database lets you skip it.
8. **Before you trust a result from an unfamiliar database, reconcile one total** with a number you already know, exactly as you did throughout section 12.15.

Interview extra point. When asked "Which SQL databases have you used?", don't just list names. Add one concrete difference you've handled: "Mostly PostgreSQL, and MySQL for our application database. The thing I watch for in MySQL is date arithmetic: subtracting dates doesn't give days, so I always use `DATEDIFF` . And its default collation ignores case, which changes how duplicate checks behave." One specific, correct detail tells the interviewer you've actually worked with the database, not just read its name. Chapter 69 explains this move, and Chapter 71 has more dialect questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Mixing <code>AND</code> / <code>OR</code> without parentheses	Too many rows; results include things you meant to exclude	Always bracket <code>OR</code> groups
<code>= NULL</code> instead of <code>IS NULL</code>	Zero rows, no error	Use <code>IS NULL</code> / <code>IS NOT NULL</code>
<code><></code> filter on a column with NULLs	Rows silently missing	Add <code>OR col IS NULL</code> , or <code>COALESCE</code> first
Fan-out from joining a finer-grain table	Counts and sums too high	Ask "what is one row?"; <code>COUNT(DISTINCT)</code> ; aggregate before joining
Summing a parent amount after joining to child rows (invoices → payments)	Invoiced or outstanding totals inflated	Aggregate the child table per parent first, then join
Storing phone numbers, PIN codes, or account numbers as numbers	Leading zeros vanish; <code>+91</code> rejected	Store identifiers as text
Answering with only part of the relevant data (due dates without payments)	"Overdue" invoices that are already paid	Ask which other tables change the answer, and join them
Not reconciling totals	A report silently loses a customer or an unassigned order	Compare report totals with a simple <code>SUM</code> on the source table
Inner join where a left join was needed	Customers or products with no activity vanish	Decide on purpose whether unmatched rows must stay

Mistake	Symptom	Fix
Right-table condition in <code>WHERE</code> after <code>LEFT JOIN</code>	Left join behaves like an inner join	Move the condition into <code>ON</code>
<code>NOT IN</code> with a subquery that can return <code>NULL</code>	Zero rows	Use <code>NOT EXISTS</code> or an anti-join
Integer division	Percentages come out as 0	Multiply by <code>1.0</code> or <code>100.0</code> first
<code>BETWEEN</code> on timestamps	Last day's records missing	Half-open range: <code>>= start AND < next_start</code>
Grouping by <code>EXTRACT(MONTH ...)</code> across years	Different years merged	<code>DATE_TRUNC('month', ...)</code>
Relying on unsorted output	Order changes between runs	Always <code>ORDER BY</code> when order matters
<code>UPDATE / DELETE</code> without <code>WHERE</code>	Every row changed	Run the <code>WHERE</code> as a <code>SELECT</code> first; use transactions
Not hand-checking a new calculation	Wrong logic ships in a report	Verify one row or one group by hand, every time

In the real world: Riverstone's monthly sales report

Every month, Riverstone's sales coordinator spends most of a day building the same report. She exports orders from the billing system to Excel, looks up customer names, filters out cancelled orders, adds up line values with discounts, and builds a pivot table by month. When a late correction arrives, she starts again.

Here is that report as one query:

```
-- Monthly sales summary
-- Revenue counts Delivered and Shipped orders (finance policy);
-- Pending and Cancelled orders are excluded.
SELECT DATE_TRUNC('month', o.order_date)::date AS month,
       COUNT(DISTINCT o.order_id)           AS orders,
       COUNT(DISTINCT o.customer_id)        AS active_customers,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue
FROM orders AS o
JOIN order_items AS oi
  ON o.order_id = oi.order_id
WHERE o.status IN ('Delivered', 'Shipped')
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

```
month | orders | active_customers | net_revenue
-----+-----+-----+-----
2026-01-01 | 3 | 3 | 104210
2026-02-01 | 5 | 5 | 161700
2026-03-01 | 2 | 2 | 31800
(3 rows)
```

It runs in well under a second, it gives the same answer every time, and a late correction just means running it again. Every idea in this chapter is in it: filtering, a join, aggregates, `COUNT(DISTINCT)` to avoid fan-out, date truncation, and a comment recording a **business rule** (which statuses count as revenue).

That business rule is the most important line in the query, and SQL can't decide it for you. Does a Shipped order count as revenue, or only Delivered? Does Pending? Finance, sales, and operations may each give a different answer. **Your first job on any report is to find out and write it down.** Chapter 24 covers how to have that conversation.

And once the query is right, nobody should have to run it by hand at all. **Chapter 20** takes this exact query and turns it into a *Daily Sales Flash*: an email that lands in every manager's inbox at 8 a.m., with the numbers in the body of the email, an alert when stock runs low, and a warning to you if the job ever fails.

Tools

- **PostgreSQL**: a free, professional-grade relational database that follows the SQL standard closely. It's the recommended database for learning and a common choice in industry.
 - **MySQL Community Server** (optional second database): the free edition of MySQL, behind a large share of web and e-commerce applications. Install the current LTS release (section 12.3).
 - **DBeaver Community Edition**: a free query editor that connects to almost any database, including both of the above. Alternatives: pgAdmin (PostgreSQL's own tool), MySQL Workbench (MySQL's own tool), or the editor built into your company's data warehouse.
 - **The Riverstone practice files** (Appendix E): `riverstone_setup.sql` (the small database used in this chapter), its MySQL twin `riverstone_setup_mysql.sql`, `ch12_queries_mysql.sql` (every query in this chapter, tested in MySQL), and the full-size version with thousands of orders, for the project.
 - **Later in the book**: a cloud data warehouse (Chapter 49), where the same SQL runs on billions of rows.
-

The project: rebuild a real report in SQL

Goal: replace one manual report with a single, commented, verified SQL query.

Option A: your own work data. Use a report you actually produce: sales, collections, inventory, attendance, leads. Get permission first, and never copy confidential data to personal devices. Remove or mask names if needed.

Option B: the full Riverstone dataset from Appendix E, if you don't have suitable work data.

Steps:

1. **Pin down the question.** Write, in one or two sentences, exactly what the report answers, for whom, and over what period. List every business rule: which statuses count, how discounts and

returns are handled, which date defines the month (order date, invoice date, or delivery date?).

2. **Map the data.** Sketch the tables you need, their keys, and the grain of each (what one row represents). Draw it on paper like Figure 12.1.
3. **Load it.** Create the tables in PostgreSQL and load the data. (For CSV files, DBeaver's import wizard works; Chapter 45 covers robust loading.) Run `COUNT(*)` on each table and compare it with the source.
4. **Build step by step.** Start with `SELECT ... FROM` one table. Add one join at a time, checking row counts after each. Add filters. Add aggregation last.
5. **Reconcile.** Compare your SQL totals with the report you trust. They *will* differ at first. Track down every difference: a cancelled order counted, a join that fanned out, a date boundary, a NULL. Write down what you found.
6. **Make it readable.** Apply the style guide from section 12.14 and comment every business rule.
7. **Save it** in a Git repository (Chapter 26) with a short README: what the report is, the rules, and how to run it.

Stretch goals:

- Add a second query for the same report broken down by salesperson or customer segment.
- Add a "data checks" query that flags problems: orders with no items, items with zero quantity, customers with no city.
- Time how long the manual process took and how long the query takes. That number belongs on your CV.
- After Chapter 20, schedule the query and deliver its result automatically by email.

You've got it when...

Check each box honestly:

- ☐ I can explain what a table, row, column, primary key, and foreign key are, using an example from my own work.
- ☐ I can look at a schema and state the grain of each table.
- ☐ I can write `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING` queries without looking up the syntax.
- ☐ I bracket every mix of `AND` and `OR` without thinking about it.
- ☐ Before filtering a column, I automatically ask whether it can be NULL.
- ☐ I choose between `INNER` and `LEFT JOIN` based on the business question, and I can write an anti-join.
- ☐ When a join is involved, I check whether counts or sums have fanned out.
- ☐ I can explain why `WHERE` can't use an aggregate or a `SELECT` alias.
- ☐ I have rebuilt one real report in SQL and reconciled it to the trusted numbers.

☐ (If you use MySQL,) I can move a query between PostgreSQL and MySQL and name the three differences that change results without an error.

When a new business question arrives and your thinking is about *the business*, not about the syntax, SQL has become a tool in your hands.

Recap

- A **relational database** stores data in strictly typed **tables**. **Primary keys** identify rows; **foreign keys** link tables; the database enforces both.
 - Data is split across tables (**normalization**) to store each fact once. **Joins** stitch it back together.
 - `SELECT` chooses columns, `FROM` names tables, `WHERE` filters rows, `ORDER BY` sorts, `LIMIT` keeps the top *n*.
 - **NULL** means unknown. Test it with `IS NULL`; remember that comparisons with NULL are never true, so `<>` filters silently drop NULL rows.
 - `CASE` adds if-then logic; `DATE_TRUNC` groups by period; text functions clean labels.
 - **Aggregates** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) plus `GROUP BY` turn rows into summaries. `HAVING` filters groups; `WHERE` filters rows.
 - **INNER JOIN** keeps matches only; **LEFT JOIN** keeps every left row. Anti-joins find what's missing. Self-joins relate a table to itself.
 - **Fan-out** from joining to a finer grain inflates counts and sums. Know the grain of every table.
 - The database runs clauses in the order **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT**, which explains most errors.
 - **Subqueries** nest queries; `NOT EXISTS` is safer than `NOT IN`. `UNION ALL` stacks results; `UNION` also removes duplicates.
 - `INSERT`, `UPDATE`, and `DELETE` change data. Check the `WHERE` first, and use **transactions**.
 - Readable SQL is correct SQL more often. Always hand-check one result.
 - **MySQL** runs the same SQL with a few different spellings and three silent traps: subtract dates with `DATEDIFF`, join text with `CONCAT`, and remember that its default collation ignores capital letters and sorts NULLs first.
-

Practice exercises

All exercises use the Riverstone database from section 12.2. Try each one before looking at the answers. For every query, **predict the number of rows first**, then run it.

Warm-up

1. List all products in the Kitchen category, cheapest first.
2. How many customers are in each segment? Show the largest segment first.

3. List the orders placed in March 2026, using a half-open date range.
4. Find customers whose name contains "Foods" or "Mart".

Core

5. Show total units sold for **every** product, including products never sold (show 0), ignoring cancelled orders. Most units first.
6. Show net revenue (non-cancelled orders) by sales rep name. Orders with no rep should appear as "Unassigned".
7. Find customers who have placed at least one order but have never had an order Delivered.
8. What is the average net order value across non-cancelled orders?
9. Show the number of payments and total amount received by payment method, largest amount first.
10. List the invoices that haven't received any payment at all, with their amount and due date, earliest due date first.

Stretch

11. For each customer who has ordered, show their first order date, most recent order date, and the number of days between them. Largest gap first.
12. Show net revenue by product category and each category's percentage of total net revenue (non-cancelled orders), to one decimal place.
13. Which customers' total net revenue is above the average customer's total net revenue?
14. For every invoice with money still owed, show the invoice amount, the number of payments, the amount paid, and the balance. Largest balance first.
15. Operations wants to chase stuck orders as of 31 March 2026: orders Pending for more than 7 days, or Shipped for more than 14 days (and not yet Delivered). Show the order, date, status, and days open, oldest first.

Think about it (no SQL needed)

16. A colleague's report says Riverstone had "19 orders" in Q1 2026. Where might that number have come from, and what's the right figure?
17. The business wants to know revenue by city. Sunrise Caterers has no city. Should the report drop it, show it as "Unknown", or something else? Who should decide?
18. Explain to a non-technical manager, in three sentences, why the sales report must not overwrite old prices when the price list changes.
19. In exercise 14 you can safely join invoices to payments and use `SUM(p.amount)`, but in section 12.10 the same join gave a wrong total. What's the difference? Use invoice 9005 in your explanation.

MySQL track (optional, section 12.16)

20. Rewrite exercise 15 (stuck orders) so it runs in MySQL and gives the same result.
21. In MySQL, show each employee as a single text column in the form `Neha Kulkarni (Sales Executive)`, next to their manager's name (or `(none)`).

22. A colleague moves a MySQL report to PostgreSQL. It used `WHERE segment = 'retail'` and `DATE '2026-03-31' - signup_date AS days_as_customer`. What happens to each line in PostgreSQL, and which result would have been wrong in MySQL all along?

Key terms

attribute/column · row/record · table · relational database · DBMS · schema · data type · `NUMERIC` vs floating point · entity-relationship (ER) diagram · primary key · composite key · foreign key · referential integrity · one-to-many · many-to-many · bridge table · normalization · grain · SQL dialect · alias · `DISTINCT` · three-valued logic · `NULL` · `COALESCE` · `CASE` · cast · aggregate function · `GROUP BY` · `HAVING` · inner join · left join · right join · full outer join · cross join · self-join · anti-join · fan-out · reconciliation · logical execution order · subquery · correlated subquery · derived table · gross margin · ageing report · `UNION` / `UNION ALL` · `INTERSECT` / `EXCEPT` · constraint · transaction · `COMMIT` / `ROLLBACK` · auto-commit · MySQL · LTS release · collation · `DATEDIFF` · `ONLY_FULL_GROUP_BY`

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 13, SQL for Real Analysis**, builds on this chapter with common table expressions, window functions (rankings, running totals, month-over-month growth), and the report patterns analysts use every week.
 - **Chapter 14, Data Cleaning & Preparation**, tackles the messy data that real databases are full of.
 - **Chapter 20, Automating Reports & Delivering Insights**, puts your queries on a schedule and delivers the results to people's inboxes, chats, and dashboards.
 - **Chapter 28** returns to databases from the designer's side: normalization rules, indexes, and query performance.
 - **Interview preparation:** the SQL Question Bank (Chapter 71) tests everything in this chapter, from "explain the difference between WHERE and HAVING" to live join and NULL puzzles, with graded model answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every query below was run against the Riverstone database, and the outputs are real.)

1.

```
SELECT product_name, unit_price
FROM products
WHERE category = 'Kitchen'
ORDER BY unit_price ASC;
```

product_name	unit_price
Water Bottle 1L	120.00
Food Container Set	650.00

(2 rows)

2.

```
SELECT segment, COUNT(*) AS num_customers
FROM customers
GROUP BY segment
ORDER BY num_customers DESC, segment;
```

segment	num_customers
Hospitality	3
Retail	3
Wholesale	2

(3 rows)

The tie between Hospitality and Retail is broken alphabetically by the second sort column. Without it, their order isn't guaranteed.

3.

```
SELECT order_id, customer_id, order_date
FROM orders
WHERE order_date >= '2026-03-01'
AND order_date < '2026-04-01'
ORDER BY order_date;
```

order_id	customer_id	order_date
5010	3	2026-03-03
5011	1	2026-03-10
5012	5	2026-03-15

(3 rows)

4.

```
SELECT customer_name
FROM customers
WHERE customer_name LIKE '%Foods%'
OR customer_name LIKE '%Mart%';
```

```
customer_name
-----
Coastal Foods
Metro Mart
(2 rows)
```

5. Aggregate the sales first, then left-join to the full product list, so the filter on order status can't remove unsold products:

```
SELECT p.product_name,
       COALESCE(s.units_sold, 0) AS units_sold
FROM products AS p
LEFT JOIN (
  SELECT oi.product_id, SUM(oi.quantity) AS units_sold
  FROM order_items AS oi
  JOIN orders AS o ON oi.order_id = o.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY oi.product_id
) AS s ON p.product_id = s.product_id
ORDER BY units_sold DESC, p.product_name;
```

```
product_name | units_sold
-----+-----
Water Bottle 1L |      230
Industrial Crate |      125
Food Container Set |       85
Storage Box 10L |       85
Storage Box 25L |       60
Garden Chair |        0
(6 rows)
```

Common wrong answer: joining `products` → `order_items` → `orders` with left joins and then writing `WHERE o.status <> 'Cancelled'`. That `WHERE` removes the Garden Chair, because its status is `NULL`. It's the "left join became an inner join" trap again.

6.

```
SELECT COALESCE(e.employee_name, 'Unassigned') AS sales_rep,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
LEFT JOIN employees AS e ON o.sales_rep_id = e.employee_id
WHERE o.status <> 'Cancelled'
GROUP BY COALESCE(e.employee_name, 'Unassigned')
ORDER BY net_revenue DESC;
```

```
sales_rep | net_revenue
-----+-----
Rahul Mehta |    146745
Farah Khan |     96660
Neha Kulkarni |    57200
Unassigned |    23325
(4 rows)
```

The join to `employees` must be a left join. An inner join would drop order 5008 and its ₹23,325.

7.

```
SELECT c.customer_name
FROM customers AS c
WHERE EXISTS (SELECT 1 FROM orders AS o
              WHERE o.customer_id = c.customer_id)
AND NOT EXISTS (SELECT 1 FROM orders AS o
                WHERE o.customer_id = c.customer_id
                  AND o.status = 'Delivered');
```

```
customer_name
-----
Northgate Distributors
(1 row)
```

Blue Bay Cafe is correctly excluded: it has never ordered at all.

8.

```
SELECT ROUND(AVG(order_revenue), 2) AS avg_order_value
FROM (
  SELECT o.order_id,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS order_revenue
  FROM orders AS o
  JOIN order_items AS oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY o.order_id
) AS per_order;
```

```
avg_order_value
-----
          29448.18
(1 row)
```

Check: total non-cancelled revenue is ₹323,930 across 11 orders; $323,930 \div 11 = 29,448.18$. ✓

9.

```
SELECT method, COUNT(*) AS payments, SUM(amount) AS amount_received
FROM payments
GROUP BY method
ORDER BY amount_received DESC;
```

method	payments	amount_received
Bank transfer	5	137960.00
UPI	4	44740.00
Cheque	1	14550.00

(3 rows)

The three amounts add up to ₹197,250, the total of the `payments` table. ✓

10. An anti-join from invoices to payments:

```
SELECT i.invoice_id, i.amount, i.due_date
FROM invoices AS i
LEFT JOIN payments AS p ON i.invoice_id = p.invoice_id
WHERE p.payment_id IS NULL
ORDER BY i.due_date;
```

```
invoice_id | amount | due_date
-----+-----+-----
      9007 | 23325.00 | 2026-03-22
      9010 | 11700.00 | 2026-04-10
(2 rows)
```

Invoice 9007 is already overdue; 9010 isn't due until April.

11.

```
SELECT c.customer_name,
       MIN(o.order_date) AS first_order,
       MAX(o.order_date) AS latest_order,
       MAX(o.order_date) - MIN(o.order_date) AS days_between
FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY days_between DESC, c.customer_name;
```

```
customer_name | first_order | latest_order | days_between
-----+-----+-----+-----
Sharma Hardware | 2026-01-05 | 2026-03-10 | 64
Green Leaf Hotels | 2026-01-20 | 2026-03-03 | 42
Metro Mart | 2026-02-06 | 2026-03-15 | 37
Coastal Foods | 2026-01-09 | 2026-02-11 | 33
Northgate Distributors | 2026-02-25 | 2026-02-25 | 0
Patel Kitchenware | 2026-01-14 | 2026-01-14 | 0
Sunrise Caterers | 2026-02-19 | 2026-02-19 | 0
(7 rows)
```

Grouping by name works here because names are unique in this data. In real data, group by `c.customer_id, c.customer_name` so two customers with the same name aren't merged.

12.

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 0) AS net_revenue,
       ROUND(100.0 * SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100))
             / (SELECT SUM(oi2.quantity * oi2.unit_price * (1 - oi2.discount_pct / 100))
                FROM order_items AS oi2
                JOIN orders AS o2 ON oi2.order_id = o2.order_id
                WHERE o2.status <> 'Cancelled'), 1) AS pct_of_total
FROM order_items AS oi
JOIN orders AS o ON oi.order_id = o.order_id
JOIN products AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category
ORDER BY net_revenue DESC;
```

category	net_revenue	pct_of_total
Industrial	161385	49.8
Storage	81810	25.3
Kitchen	80735	24.9

(3 rows)

Notice the `100.0`, which avoids integer division. The scalar subquery repeats the cancelled-order filter; if it didn't, the percentages would be calculated against a different total and wouldn't add up to 100. In Chapter 13, a window function does this far more neatly: `SUM(...) OVER ()`.

13.

```
SELECT customer_name, ROUND(customer_revenue, 0) AS customer_revenue
FROM (
    SELECT c.customer_name,
           SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS customer_revenue
    FROM customers AS c
    JOIN orders      AS o  ON c.customer_id = o.customer_id
    JOIN order_items AS oi ON o.order_id    = oi.order_id
    WHERE o.status <> 'Cancelled'
    GROUP BY c.customer_name
) AS t
WHERE customer_revenue > (
    SELECT AVG(customer_revenue)
    FROM (
        SELECT o.customer_id,
               SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS customer_revenue
        FROM orders AS o
        JOIN order_items AS oi ON o.order_id = oi.order_id
        WHERE o.status <> 'Cancelled'
        GROUP BY o.customer_id
    ) AS x
)
ORDER BY customer_revenue DESC;
```

customer_name	customer_revenue
Coastal Foods	105885
Northgate Distributors	76560

(2 rows)

The average customer total is ₹46,276. Notice how much the logic repeats itself. That repetition is exactly the problem CTEs solve in Chapter 13, where this becomes a short, readable query.

14.

```
SELECT i.invoice_id,
       i.amount,
       COUNT(p.payment_id) AS num_payments,
       COALESCE(SUM(p.amount), 0) AS amount_paid,
       i.amount - COALESCE(SUM(p.amount), 0) AS balance
FROM invoices AS i
LEFT JOIN payments AS p ON i.invoice_id = p.invoice_id
GROUP BY i.invoice_id, i.amount
HAVING i.amount - COALESCE(SUM(p.amount), 0) > 0
ORDER BY balance DESC;
```

invoice_id	amount	num_payments	amount_paid	balance
9008	76560.00	1	30000.00	46560.00
9007	23325.00	0	0	23325.00
9006	32625.00	1	20000.00	12625.00
9010	11700.00	0	0	11700.00
9003	16250.00	1	10000.00	6250.00

(5 rows)

`COUNT(p.payment_id)` counts only real payments, so unpaid invoices show 0; `COUNT(*)` would wrongly show 1 for them, because the left join still produces one row. The balances add up to ₹100,460. ✓

15.

```
SELECT order_id,
       order_date,
       status,
       DATE '2026-03-31' - order_date AS days_open
FROM orders
WHERE (status = 'Pending' AND DATE '2026-03-31' - order_date > 7)
      OR (status = 'Shipped' AND DATE '2026-03-31' - order_date > 14)
ORDER BY days_open DESC;
```

order_id	order_date	status	days_open
5009	2026-02-25	Shipped	34
5011	2026-03-10	Shipped	21
5012	2026-03-15	Pending	16

(3 rows)

The parentheses matter: each status has its own day limit (section 12.5). Order 5009 has been "Shipped" for 34 days, which almost certainly means someone forgot to mark it Delivered, or it's lost. Either way, somebody should find out today.

16. 19 is the number of rows in `order_items`, i.e. order lines, probably from counting rows after a join or from the wrong table. Riverstone had **12 orders** in Q1 2026 (11 if the cancelled order is excluded; which one to report is a business rule to agree on). It's the fan-out trap in the wild.

17. Dropping Sunrise Caterers would understate total revenue by ₹23,325, so total revenue by city wouldn't match total revenue overall. Showing "Unknown" keeps the totals reconciled and makes the data-quality gap visible. The business decides how to present it, but the analyst's job is to flag it, and ideally to get the missing city fixed at the source.

18. A sample answer: "Each sale records the price the customer actually paid on that day. If we used today's price list to calculate old sales, every price change would rewrite our history, and last quarter's revenue would keep changing. So we store the price at the moment of sale, and the price list only affects new orders."

19. In exercise 14, the query groups by invoice and uses `i.amount` only as a grouping column: it's never summed, so repeating it on two rows does no harm. In section 12.10, the query summed `i.amount` across the joined rows. Invoice 9005 (₹14,640) was paid in two instalments, so the join produced two rows for it, and `SUM(i.amount)` counted ₹29,280. **Repeated rows are only a problem when you add up a column from the table that got repeated.**

20. (MySQL.) Replace each date subtraction with `DATEDIFF`, later date first:

```
SELECT order_id,
       order_date,
       status,
       DATEDIFF(DATE '2026-03-31', order_date) AS days_open
FROM orders
WHERE (status = 'Pending' AND DATEDIFF(DATE '2026-03-31', order_date) > 7)
      OR (status = 'Shipped' AND DATEDIFF(DATE '2026-03-31', order_date) > 14)
ORDER BY days_open DESC;
```

order_id	order_date	status	days_open
5009	2026-02-25	Shipped	34
5011	2026-03-10	Shipped	21
5012	2026-03-15	Pending	16

The same three orders as the PostgreSQL answer. If you had left `DATE '2026-03-31' - order_date` in place, MySQL would have compared numbers like `20260331 - 20260315 = 16` for order 5012 (correct only by luck, because both dates are in the same month) and `20260331 - 20260225 = 106` for order 5009, instead of 34.

21. (MySQL.) `CONCAT` joins any number of pieces; `||` would return 0:

```
SELECT CONCAT(e.employee_name, ' (', e.job_title, ')') AS employee,
       COALESCE(m.employee_name, '(none)') AS reports_to
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

employee	reports_to
Anita Rao (Sales Head)	(none)
Vikram Singh (Sales Manager)	Anita Rao
Neha Kulkarni (Sales Executive)	Vikram Singh
Rahul Mehta (Sales Executive)	Vikram Singh
Farah Khan (Sales Executive)	Anita Rao

This exact query also runs in PostgreSQL. One detail: in MySQL, `CONCAT` returns NULL if *any* piece is NULL, so an employee with no job title would show a blank. `CONCAT_WS(' ', ...)` or `COALESCE` on each piece avoids that.

22. `WHERE segment = 'retail'` found the three Retail customers in MySQL, because MySQL's default collation ignores case, but returns **zero rows** in PostgreSQL, silently. Fix it with `'Retail'` or `LOWER(segment) = 'retail'`. The date line behaves the other way: PostgreSQL gives the correct number of days, while MySQL was producing meaningless numbers (it subtracted dates as if they were numbers like `20260331`). So the report's "days as customer" figures were **wrong all along in MySQL**, and nobody noticed because no error appeared. Moving between databases is a good moment to reconcile every number against a trusted total.